DIPARTIMENTO DI MATEMATICA

CORSO DI LAUREA IN MATEMATICA

TESI DI LAUREA
IN
METODI NUMERICI IN DATA SCIENCE

Tecniche probabilistiche per il riconoscimento di argomenti latenti: il modello PLSA

RELATRICE:
Prof. Nicoletta Del Buono

LAUREANDO:
Angelo Gabriele Lapacciana

ANNO ACCADEMICO 2024 - 2025

"La passione è un ponte tra le possibilità e le probabilità."

— T. F. Hodge

Dedico questo lavoro a me stesso, perché troppe volte ho sottovalutato il mio impegno nei traguardi raggiunti e sono stato eccessivamente severo nei miei confronti. Senza la costanza e la dedizione che mi hanno guidato, non sarei arrivato fin qui. Oggi, invece, scelgo di riconoscere a me stesso il merito di avercela fatta.

Indice

Introduzione 1

1 Fondamenti teorici e tecnici per la Probabilistic Latent Semantic Analysis 3

1.1 Natural Language Processing e rappresentazione del testo 3

1.1.1 Natural Language Processing 4

1.1.2 Applicazioni del Natural Language Processing 4

1.1.3 Come funziona il Natural Language Processing? 5

1.1.4 Vector Space Model 7

1.1.5 Costruzione della matrice termine-documento 9

1.1.6 Information Retrieval 9

1.2 Strumenti di algebra lineare 11

1.2.1 Singular Value Decomposition 11

1.2.2 SVD troncata e matrice di migliore approssimazione 12

1.3 Fondamenti di probabilità 13

1.3.1 Spazi di probabilità 14

1.3.2 Distribuzioni 15

1.3.3 Probabilità condizionata e congiunta 16

1.4 Modelli statistici e di apprendimento 18

1.4.1 Modello generativo 18

1.4.2 Apprendimento non supervisionato e clustering documentale 19

1.4.3 Apprendimento statistico: overfitting, underfitting, annealing e early stopping 20

1.4.4 Divergenza di Kullback–Leibler e cross-entropy 22

2 Probabilistic Latent Semantic Analysis 25
2.1 Latent Semantic Analysis 25
2.1.1 Analisi Semantica Latente tramite Decomposizione ai Valori Singolari 27
2.2 Analisi Probabilistica Semantica Latente 28
2.2.1 Aspect Model 29
2.2.2 Addestramento del modello con l'algoritmo EM 33
2.2.3 Ottimizzazione del modello e generalizzazione: il Tempered EM 34
2.2.4 Geometria dell'Aspect Model e Spazio Semantico Latente Probabilistico 36
2.3 Una visione moderna del LSA 38
2.3.1 Chiarimenti sull'algoritmo EM 42

3 Confronti sperimentali e applicazioni della PLSA 45
3.1 Confronto tra PLSA e LSA 45
3.1.1 Confronto geometrico 46
3.1.2 Confronto applicativo: valutazione della *perplexity* 47
3.1.3 Information retieval con PLSA e LSA 50
3.2 Applicazione della PLSA alla Computer Vision 57
3.2.1 Auto-annotazione 59
3.3 PLSA su GPU: un approccio efficiente 60
3.3.1 Accelerazione dell'algoritmo EM tramite moltiplicazioni matrice-vettore 60
3.3.2 Sfruttare la sparsità dei dati 63
3.3.3 Ottimizzazione sull'architettura Fermi 63
3.3.4 Esperimenti 64

Bibliografia 69

Introduzione

Negli ultimi decenni la rapida crescita della quantità di dati testuali disponibili ha reso sempre più centrale lo sviluppo di strumenti matematici e computazionali per l'analisi automatica dei documenti. In particolare, l'ambito del *Natural Language Processing* (NLP) e quello dell'*Information Retrieval* (IR) hanno posto al centro della loro ricerca la rappresentazione semantica dei testi e la capacità di estrarne conoscenza utile, riducendo al contempo la complessità delle informazioni trattate.

All'interno di questo contesto, modelli come la *Latent Semantic Analysis* (LSA) e la sua estensione probabilistica, la *Probabilistic Latent Semantic Analysis* (PLSA), rivestono un ruolo fondamentale. Essi consentono infatti di mappare documenti e termini in spazi latenti a bassa dimensionalità, catturando relazioni semantiche non immediatamente osservabili nella rappresentazione originale. La PLSA, in particolare, introduce una cornice statistica che permette una maggiore flessibilità nell'interpretazione dei dati e una migliore generalizzazione in compiti applicativi complessi.

La presente tesi si propone di fornire una trattazione organica della PLSA, dalle sue fondamenta teoriche alle applicazioni pratiche, mettendone in luce le principali caratteristiche e confrontandola con modelli affini. Il lavoro si colloca pertanto come sintesi dei risultati presenti in letteratura, con particolare attenzione agli aspetti matematici che ne costituiscono la base e agli esperimenti che ne hanno dimostrato l'efficacia.

La tesi è strutturata come segue:

- nel **Capitolo 1** vengono presentati i concetti preliminari utili alla comprensione del modello PLSA, tra cui elementi di NLP, algebra lineare, probabilità e apprendimento statistico;

- il **Capitolo 2** è dedicato alla trattazione teorica della PLSA: dopo un richiamo alla LSA, vengono introdotti l'*Aspect Model*, l'algoritmo EM e le sue varianti, la geometria del modello e una visione moderna del legame tra PLSA e LSA;
- nel **Capitolo 3** si analizzano i confronti sperimentali e le applicazioni pratiche della PLSA, riportando risultati numerici tratti dalla letteratura e illustrando casi d'uso significativi, tra cui il recupero di informazione, la *Computer Vision* e l'implementazione su GPU.

Questa struttura consente di partire dalle basi teoriche e giungere progressivamente agli sviluppi più avanzati e agli utilizzi concreti del modello, mantenendo un filo conduttore che unisce matematica, statistica e applicazioni.

Capitolo 1

Fondamenti teorici e tecnici per la Probabilistic Latent Semantic Analysis

In questo capitolo vengono presentati alcuni concetti teorici che, pur essendo collegati alla *Probabilistic Latent Semantic Analysis*, risultano marginali rispetto al nucleo centrale della trattazione. Si è scelto pertanto di raccoglierli separatamente, così da non appesantire il capitolo principale, ma offrire comunque al lettore un quadro più completo delle basi teoriche del modello di interesse di questa tesi.

1.1 Natural Language Processing e rappresentazione del testo

L'analisi automatica del linguaggio naturale è alla base di numerose applicazioni moderne, tra cui l'Information Retrieval e il Text Mining. Per consentire l'elaborazione computazionale dei documenti testuali, è necessario trasformare il testo, originariamente non strutturato, in rappresentazioni numeriche adatte a essere trattate da algoritmi statistici e modelli matematici.

In questa sezione verranno introdotti i concetti fondamentali del Natural Language Processing, alcune delle sue principali applicazioni e il suo stretto legame

con i sistemi di Information Retrieval, che costituiscono il quadro operativo in cui molte tecniche di rappresentazione del testo trovano applicazione.

Saranno inoltre presentati i metodi classici di rappresentazione, di documenti quali il modello *Bag-of-Words* e il calcolo dei pesi tramite *Term Frequency–Inverse Document Frequency* (TF–IDF). Queste tecniche forniscono la base del Vector Space Model (VSM), che rappresenta un punto di partenza essenziale per lo sviluppo sia di sistemi di recupero automatico dell'informazione sia di tecniche più avanzate di riduzione dimensionale e modellizzazione probabilistica, che verranno affrontate nei capitoli successivi della tesi.

1.1.1 Natural Language Processing

Il **Natural Language Processing** (NLP) è la disciplina che si occupa di costruire macchine in grado di manipolare il linguaggio umano nel modo in cui è scritto, parlato e organizzato [1]. Si è evoluta dalla linguistica computazionale, che utilizza l'informatica per comprendere i principi del linguaggio, ma piuttosto che sviluppare quadri teorici, il NLP è una disciplina ingegneristica che mira a sviluppare tecnologie per svolgere compiti utili. Il NLP può essere suddiviso in due sottocampi sovrapposti: il *Natural Language Understanding* (NLU), che si concentra sull'analisi semantica o sulla determinazione del significato previsto di un testo, e il *Natural Language Generation* (NLG), che si concentra sulla generazione di testo da parte di una macchina. Il NLP è separato dal riconoscimento vocale, ma spesso è utilizzato insieme ad esso, che cerca di analizzare il linguaggio parlato in parole, trasformando il suono in testo e viceversa.

1.1.2 Applicazioni del Natural Language Processing

Il NLP è impiegato per un'ampia gamma di attività collegate all'elaborazione del linguaggio, tra cui analisi del sentiment, la traduzione automatica, il riconoscimento di entità, la risposta automatica (question answering) e l'uso e lo sviluppo dei chatbot.

Queste tecniche permettono ai sistemi informatici di comprendere, interpretare e generare contenuti testuali o parlati in linguaggio umano, supportando applicazioni quotidiane come motori di ricerca, assistenti virtuali (Alexa, Siri, Cortana) e

traduzione automatica.

In ambito aziendale, il NLP contribuisce a migliorare la produttività e a ottimizzare i processi organizzativi, mentre nel settore della ricerca e della gestione delle informazioni viene utilizzato per il recupero, l'analisi e la sintesi di grandi volumi di dati testuali. Complessivamente, il NLP rappresenta una tecnologia chiave per lo sviluppo di sistemi intelligenti capaci di interagire in modo naturale con l'uomo.

1.1.3 Come funziona il Natural Language Processing?

I modelli di NLP funzionano individuando relazioni tra le componenti del linguaggio, quali lettere, parole e frasi presenti in un dataset di testo. Le architetture di NLP utilizzano metodi per la pre-elaborazione dei dati, l'estrazione delle feature e la modellazione.

Pre-elaborazione dei dati: Prima che un modello elabori il testo per un'attività specifica, è spesso necessario pre-elaborarlo per migliorare le prestazioni o convertire parole e caratteri in un formato comprensibile. Tecniche comuni includono:

- **Stemming e lemmatization:** lo stemming riduce le parole alle loro forme base usando regole euristiche, mentre la lemmatization analizza la morfologia della parola per trovarne la radice.
- **Segmentazione delle frasi:** suddivide il testo in unità linguisticamente significative, gestendo correttamente abbreviazioni e delimitatori di frase.
- **Rimozione delle stop word:** elimina le parole più comuni che non aggiungono informazioni rilevanti. Ad esempio, "il", "un", "uno" e così via.
- **Tokenizzazione:** suddivide il testo in singole parole e frammenti di parole, convertendo le parole in token numerici per l'input ai modelli di deep learning.

Estrazione delle feature: Le tecniche di apprendimento automatico convenzionali lavorano su feature numeriche che rappresentano i documenti. Alcune tecniche includono:

- **Bag-of-Words:** conta il numero di volte in cui ogni parola o n-gramma (combinazione di n parole) appare in un documento. Per esempio, di seguito, il

modello Bag-of-Words crea una rappresentazione numerica del set di dati in base al numero di parole presenti nel documento in ogni indice di parole.

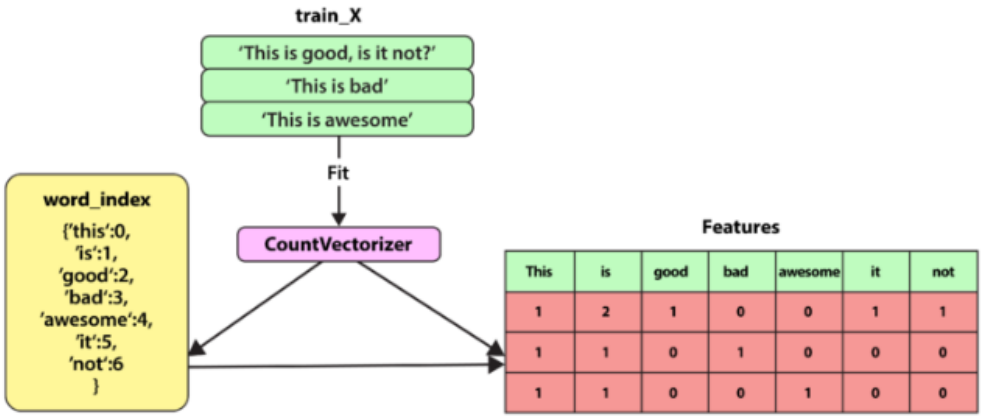


Figura 1.1: Dato un corpus di documenti, un *tokenizer* mappa ogni parola a un indice. Quindi può tradurre qualsiasi documento in una sequenza di numeri [1]

- **TF-IDF**(*Term Frequency–Inverse Document Frequency*)**:** pesa le parole in base alla loro importanza nel documento e nel corpus. La frequenza del termine (TF) è il numero di occorrenze diviso il numero totale di parole, mentre la frequenza inversa del documento (IDF) è calcolata come:

$$IDF(t) = \log \frac{N}{n_t}$$

  dove N è il numero totale di documenti e n_t è il numero di documenti contenenti il termine t.
  Una parola è importante se ricorre molte volte in un documento. Ma questo crea un problema. Parole come "un" e "il" compaiono spesso e come tali, il loro punteggio TF sarà sempre alto. Risolviamo questo problema utilizzando la Frequenza Inversa del Documento, che è alta se la parola è rara e bassa se è comune in tutto il corpus. Il punteggio TF-IDF di un termine è il prodotto di TF e IDF.

- **Word embeddings:** Word2Vec e GloVe [1] rappresentano le parole come vettori densi, catturando le relazioni semantiche e contestuali tra le parole.

Modellazione: dopo la fase di pre elaborazione, i dati vengono trasformati in una rappresentazione adatta all'elaborazione da parte di modelli NLP. Tra i principali approcci si possono distinguere:

- **Modelli linguistici**: mirano a stimare la probabilità di una sequenza di parole e a predire l'elemento successivo dato il contesto. Un esempio è fornito dai modelli probabilistici basati sull'ipotesi di Markov:

$$P(W_n) = P(W_n|W_{n-1})$$

- **Reti neurali profonde**: consentono di apprendere automaticamente rappresentazioni del testo senza necessità di caratteristiche estratte manualmente, anche se metodi come TF-IDF o Bag-of-Words possono comunque essere utilizzati come input.

1.1.4 Vector Space Model

Le tecniche di Natural Language Processing mirano dunque a trasformare il linguaggio naturale, originariamente non strutturato, in rappresentazioni numeriche. Un modo particolarmente influente per formalizzare queste rappresentazioni è il *Vector Space Model* (VSM), che descrive i documenti come vettori in uno spazio multidimensionale e fornisce le basi geometriche per il calcolo della similarità tra testi.

Il **Vector Space Model** è un modello algebrico per rappresentare documenti di testo (o più in generale, elementi) come vettori in modo tale che la distanza tra i vettori rappresenti la pertinenza tra i documenti [2]. Viene utilizzato nel filtraggio delle informazioni, nell'Information Retrieval, nell'indicizzazione e nelle classifiche di pertinenza.

La definizione di *termine* dipende dall'applicazione. In genere, i termini possono essere singoli vocaboli, espressioni chiave o frasi più lunghe. Se si scelgono i vocaboli come termini, la dimensionalità del vettore corrisponde al numero di elementi presenti nel vocabolario (ossia il totale delle unità lessicali distinte del corpus).

Le operazioni vettoriali possono essere utilizzate per confrontare documenti con query. I documenti candidati provenienti da un corpus possono essere recuperati e classificati attraverso diversi metodi. Nel VSM la pertinenza di un documento

rispetto a una query viene stimata calcolando la similarità tra il vettore che rappresenta il documento e quello che rappresenta la query. In particolare, la query viene modellata come un vettore con le stesse dimensioni dei vettori documentali, consentendo un confronto diretto.

La similarità può essere misurata calcolando il coseno dell'angolo θ tra i due vettori:

$$\cos(\theta) = \frac{\mathbf{d_j} \cdot \mathbf{q}}{\|\mathbf{d_j}\| \|\mathbf{q}\|}$$

dove $\mathbf{d_j} \cdot \mathbf{q}$ rappresenta l'intersezione (cioè il prodotto scalare) tra il vettore del documento d_j e quello della query q, mentre $\|\mathbf{d_j}\|$ e $\|\mathbf{q}\|$ indicano rispettivamente le loro norme euclidee:

$$\|\mathbf{q}\| = \sqrt{\sum_{i=1}^{N} q_i^2}.$$

Il valore della similarità coseno varia tra 0 e 1, avendo solo componenti non negative: un valore pari a 0 indica che i vettori sono ortogonali (assenza di corrispondenza tra documento e query), mentre valori più prossimi a 1 indicano un'elevata somiglianza.

Il VSM presenta numerosi vantaggi rispetto al modello booleano classico:

- consente di ordinare i documenti in base alla loro pertinenza stimata, anziché restituire semplicemente un insieme non ordinato;
- permette di recuperare documenti che condividono solo parzialmente i termini della query;
- garantisce una rappresentazione più densa e flessibile dell'intera collezione documentale.

Tuttavia, esso presenta anche alcune limitazioni. In primo luogo, assume l'indipendenza dei termini, il che rende difficile rappresentare correttamente le frasi e le relazioni semantiche. Inoltre, due documenti con significato simile ma vocabolario differente possono risultare scarsamente correlati. Molti di questi problemi possono essere mitigati integrando strumenti aggiuntivi, quali la Decomposizione ai Valori Singolari (SVD).

1.1.5 Costruzione della matrice termine-documento

Una volta introdotto il VSM, la sua realizzazione operativa passa attraverso la costruzione della **matrice termine-documento**. Si tratta di una rappresentazione tabellare in cui le *righe* corrispondono ai termini del vocabolario e le *colonne* ai documenti della collezione. L'elemento a_{ij} della matrice indica il peso associato al termine t_i nel documento d_j.

A seconda del criterio di pesatura scelto, i valori a_{ij} possono assumere diverse forme:

- **conteggi grezzi** (*term frequency*), ovvero il numero di volte in cui il termine t_i compare in d_j;
- **pesi normalizzati**, come la *term frequency-inverse document frequency* (TF–IDF), che tengono conto sia della frequenza del termine nel documento sia della sua distribuzione nell'intera collezione;
- **valori binari**, che indicano semplicemente la presenza o assenza di un termine in un documento.

Formalmente, data una collezione di m termini e n documenti, la matrice termine-documento ha dimensione $m \times n$ e può essere scritta come:

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}.$$

Questa rappresentazione fornisce la base per numerose applicazioni: dal calcolo della similarità tra documenti all'Information Retrieval, fino alle tecniche di **riduzione dimensionale** e **modellizzazione probabilistica** che verranno discusse nelle sezioni successive.

1.1.6 Information Retrieval

Il Vector Space Model costituisce la base concettuale di numerose tecniche di **Information Retrieval**. In particolare, rappresentando documenti e query in uno

spazio multidimensionale, il VSM permette di calcolare la loro similarità e quindi di ordinare i documenti in base alla loro rilevanza rispetto a una determinata query.

In questo senso, l'Information Retrieval può essere visto come un'applicazione pratica del VSM: mentre il modello fornisce il formalismo matematico per rappresentare e confrontare testi, l'IR ne sfrutta le potenzialità per risolvere problemi reali, come la ricerca di informazioni sul web, la classificazione di documenti o il filtraggio di contenuti.

L'Information Retrieval ha come obiettivo l'identificazione e il reperimento delle risorse pertinenti a una determinata esigenza informativa. Quest'ultima viene generalmente espressa attraverso una query, che può essere interpretata dal sistema in base a contenuti testuali (ad esempio con indicizzazione full-text) o ad altre caratteristiche dei documenti. L'IR non riguarda soltanto la ricerca di informazioni all'interno dei documenti, ma anche l'individuazione dei documenti stessi, dei metadati descrittivi e di risorse eterogenee come database di testi, immagini o suoni.

Poiché l'IR nasce per affrontare il problema del sovraccarico informativo, la valutazione delle prestazioni di un sistema di recupero non si limita alla capacità di restituire risultati, ma si concentra sulla loro qualità. In questo contesto entrano in gioco due metriche fondamentali:

- **Precision**, che misura la proporzione di documenti rilevanti all'interno di quelli restituiti dal sistema;

- **Recall**, che quantifica invece la proporzione di documenti rilevanti effettivamente recuperati rispetto al totale dei documenti pertinenti esistenti.

L'equilibrio tra precision e recall è cruciale: un sistema che restituisce pochi documenti altamente pertinenti ottiene un'alta precisione ma può avere un basso recall, mentre uno che restituisce molti documenti aumenta il recall ma rischia di ridurre la precisione. I moderni sistemi di IR, come i motori di ricerca web, mirano a ottimizzare questo compromesso, bilanciando pertinenza e copertura per soddisfare al meglio le esigenze degli utenti.

1.2 Strumenti di algebra lineare

Gran parte dei modelli per l'analisi semantica latente, inclusa la PLSA, si basano su strumenti dell'algebra lineare. In particolare, le matrici derivate dal testo (matrice termine-documento, matrici di co-occorrenza) possono essere analizzate ed approssimate tramite fattorizzazioni che consentono di ridurre la complessità computazionale e di evidenziare strutture latenti.

In questa sezione verranno introdotti i concetti fondamentali alla base di tali metodi. Si partirà dalla Singular Value Decomposition (SVD), per poi discutere la sua forma troncata e il ruolo del teorema di Eckart–Young nella costruzione della migliore approssimazione possibile di una matrice a rango ridotto.

1.2.1 Singular Value Decomposition

Dopo aver introdotto i modelli di rappresentazione del testo e la costruzione della matrice termine–documento, è naturale domandarsi come poter analizzare o semplificare tali strutture ad alta dimensionalità. Le matrici che rappresentano i dati testuali, infatti, sono in genere di dimensioni molto grandi e caratterizzate da forte sparsità, rendendo complesso sia l'analisi diretta sia l'impiego in modelli di apprendimento.

In questo contesto riveste un ruolo centrale la **Singular Value Decomposition** (SVD), una fattorizzazione matriciale che permette di rappresentare una qualunque matrice come il prodotto di due matrici ortogonali e una diagonale.

Teorema 1.2.1 (Teorema di esistenza della SVD)**.** *Sia $X \in \mathbb{R}^{n\times m}$ con rango $k \leq min(n, m)$. Allora esistono $U = [u_1, \dots, u_n] \in \mathbb{R}^{n\times n}$, $V = [v_1, \dots, v_m] \in \mathbb{R}^{m\times m}$ ortogonali e $\Sigma \in \mathbb{R}^{n\times m}$ diagonale con valori singolari positivi $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k > 0$, tali che*

$$X = U\Sigma V^\top$$

La struttura di Σ è:

$$\Sigma = \begin{pmatrix} \sigma_1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_k & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \end{pmatrix}$$

mentre i vettori colonna di U e di V sono detti rispettivamente **vettori singolari sinistri** e **destri**. Inoltre la matrice X può essere scritta come

$$X = \sum_{i=1}^{r} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^\top,$$

dove $\mathbf{u}_i$ e $\mathbf{v}_i$ sono rispettivamente i *vettori singolari* a sinistra e a destra associati a σ_i, detti *valori singolari*.

1.2.2 SVD troncata e matrice di migliore approssimazione

Alcune applicazioni pratiche richiedono di risolvere il problema dell'*approssimazione* di una matrice X con un'altra matrice $\bar{X}$, detta *troncata*, di rango k.

Il Teorema di Eckart–Young fornisce una proprietà ottimale della SVD nel rappresentare una matrice con una sua approssimazione di rango ridotto. Questo è cruciale in contesti come la riduzione dimensionale, la rimozione di rumore o la compressione di dati.

Teorema 1.2.2 (Eckart–Young)**.** *Sia $X \in \mathbb{R}^{n \times m}$ con fattorizzazione a valori singolari $X = U\Sigma V^\top$, e siano $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k > 0$ i suoi valori singolari, dove $k = rank(X)$.*

Per ogni $r < k$, la matrice:

$$X_r = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top = U_r \Sigma_r V_r^\top$$

è l'approssimazione di rango r che minimizza l'errore rispetto a X, sia in norma di Frobenius che in norma spettrale:

- *In norma di Frobenius:*

$$\|X - X_r\|_F^2 = \min_{\substack{B \in \mathbb{R}^{n \times m} \\ rank(B)=r}} \|X - B\|_F^2 = \sum_{i=r+1}^{k} \sigma_i^2$$

- *In norma spettrale:*

$$\|X - X_r\|_2^2 = \min_{\substack{B \in \mathbb{R}^{n \times m} \\ rank(B)=r}} \|X - B\|_2^2 = \sigma_{r+1}$$

Questo risultato garantisce che troncare la SVD mantenendo solo i primi r valori singolari fornisce la miglior approssimazione possibile in termini di errore quadratico medio (norma di Frobenius) e di errore massimo (norma spettrale). È su questa proprietà che si basano molte tecniche moderne di machine learning e data compression. La possibilità di approssimare, in modo ottimale, una matrice arbitraria con una di rango "basso" permette la compressione di grandi insiemi di dati, tipicamente affetti da errore (o "rumore").

1.3 Fondamenti di probabilità

La Probabilistic Latent Semantic Analysis (PLSA) si basa su una rappresentazione probabilistica dei dati testuali: i documenti vengono considerati come realizzazioni di variabili aleatorie generate da distribuzioni sottostanti. Per comprendere questo approccio è necessario richiamare alcuni concetti fondamentali della teoria della probabilità, che forniscono il linguaggio e gli strumenti matematici alla base della modellizzazione statistica.

In questa sezione vengono introdotti:

- il formalismo degli spazi di probabilità, che definisce la struttura di base su cui si costruiscono i modelli;
- le distribuzioni, con particolare attenzione al caso multinomiale, essenziale per descrivere la frequenza delle parole nei documenti;
- la probabilità condizionata e congiunta, indispensabili per modellare le relazioni tra variabili e per comprendere il legame con l'inferenza bayesiana.

1.3.1 Spazi di probabilità

Uno **spazio di probabilità** è una terna

$$(\Omega, \mathcal{F}, P)$$

che rappresenta un modello matematico per un esperimento o una classe di situazioni reali [3]. In particolare:

- L'insieme Ω, detto **spazio campionario**, comprende tutti i possibili esiti dell'esperimento. Un esito è il risultato di una singola realizzazione e ogni esecuzione dell'esperimento deve produrre esattamente un risultato. Se due esiti differiscono in un modo rilevante ai fini dell'analisi, essi sono considerati distinti.

- La **σ-algebra** $\mathcal{F}$ è una collezione di sottoinsiemi di Ω, chiamati *eventi*. Un evento si considera accaduto se l'esito osservato appartiene a quell'insieme. Poiché lo stesso esito può appartenere a più eventi, è possibile che più eventi si verifichino contemporaneamente. Formalmente, $\mathcal{F} \subseteq 2^{\Omega}$ e deve soddisfare le proprietà seguenti:

  1. $\Omega \in \mathcal{F}$;
  2. $\mathcal{F}$ è chiuso rispetto ai *complementari*: se $A \in \mathcal{F}$, allora $(\Omega \setminus A) \in \mathcal{F}$;
  3. $\mathcal{F}$ è chiuso rispetto a *unione numerabile*: se $A_i \in \mathcal{F}$ per $i = 1,2,\ldots$, allora $(\bigcup_{i=1}^{\infty} A_i) \in \mathcal{F}$.

- La **funzione di misura di probabilità** $P : \mathcal{F} \to [0,1]$ assegna a ciascun evento un numero reale tra 0 e 1 e deve rispettare:

  1. la *additività numerabile*: se $\{A_i\}_{i=1}^{\infty} \subseteq \mathcal{F}$ è una collezione numerabile di eventi mutuamente disgiunti, allora

  $$P\Big(\bigcup_{i=1}^{\infty} A_i\Big) = \sum_{i=1}^{\infty} P(A_i);$$

  2. la probabilità dell'intero spazio campionario è 1:

  $$P(\Omega) = 1,$$

garantendo che un esito dell'esperimento si verifichi sicuramente.

Non tutti i sottoinsiemi di Ω devono necessariamente essere eventi: alcuni possono non essere di interesse, altri possono essere non "misurabili".

1.3.2 Distribuzioni

Partendo dalla struttura formale degli spazi di probabilità, le **distribuzioni** forniscono una descrizione quantitativa di come i possibili risultati di una variabile casuale si verificano. In altre parole, mentre lo spazio campionario definisce l'insieme degli esiti e le regole generali per l'assegnazione delle misure, le distribuzioni ne indicano la ripartizione specifica tra i singoli valori o intervalli.

Le distribuzioni possono riguardare variabili casuali discrete o continue. Per le variabili continue, si utilizza la **funzione di distribuzione cumulativa** $F_X(x) = P(X \leq x)$, che rappresenta la probabilità cumulata fino a un certo valore x. Tale funzione corrisponde all'area sottesa dalla **funzione di densità** $f_X(x)$ sull'intervallo $(-\infty, x]$, come illustrato in Figura 1.2.

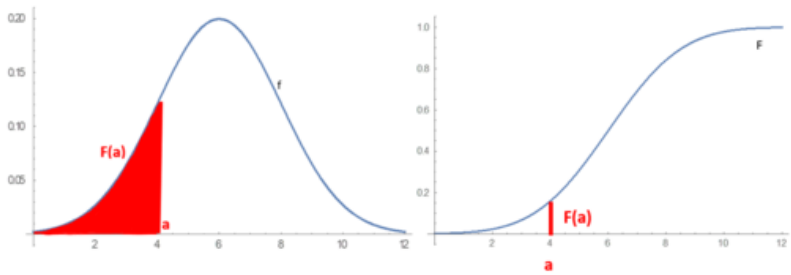


Figura 1.2: A sinistra, una funzione di densità di probabilità; a destra, la corrispondente funzione cumulativa. Il valore **a** della funzione cumulativa corrisponde all'area sotto la curva di densità fino al punto **a** [4]

In generale, le distribuzioni continue sono completamente descritte dalla loro funzione di densità, che quantifica la probabilità infinitesimale associata a ciascun valore della variabile casuale. Per le variabili discrete, invece, la funzione assegna una misura ai singoli esiti, spesso in forma di conteggi o frequenze relative.

Un esempio rilevante di distribuzione discreta, utilizzato nel capitolo principale di questo lavoro, è la **distribuzione multinomiale** [5]. Essa estende la distribuzione binomiale e di Bernoulli a esperimenti con più categorie possibili per prova. Per n prove indipendenti, ciascuna delle quali produce un risultato in una delle k

categorie con probabilità fissa $p_1, \ldots, p_k$, la multinomiale descrive la probabilità di osservare una particolare combinazione di conteggi $(X_1, \ldots, X_k)$.

Casi particolari della distribuzione multinomiale includono:

- $k = 2$, $n = 1$: distribuzione di Bernoulli;
- $k = 2$, $n > 1$: distribuzione binomiale;
- $k > 2$, $n = 1$: distribuzione categoriale.

Formalmente, siano X_i le variabili casuali che contano il numero di volte in cui ciascun esito i si verifica nelle n prove. Con probabilità associate $p_1, \ldots, p_k$ tali che $p_i \geq 0$ e $\sum_{i=1}^{k} p_i = 1$, il vettore $X = (X_1, \ldots, X_k)$ segue una distribuzione multinomiale con parametri n e $p = (p_1, \ldots, p_k)$. Sebbene le prove siano indipendenti, le variabili X_i non lo sono, poiché la loro somma è vincolata a n.

1.3.3 Probabilità condizionata e congiunta

Le distribuzioni di probabilità descritte finora riguardano singole variabili casuali, sia discrete sia continue. Tuttavia, nella pratica e nella modellizzazione matematica è spesso necessario considerare più variabili simultaneamente e analizzare le relazioni che intercorrono tra esse. In questo contesto assumono particolare importanza le **probabilità congiunte**, che descrivono la distribuzione combinata di più variabili, e le **probabilità condizionate**, che permettono di aggiornare le stime sull'occorrenza di un evento o di una variabile in funzione dell'informazione disponibile su un'altra. Questi concetti estendono in modo naturale la nozione di distribuzione e costituiscono la base per l'inferenza statistica e per numerosi modelli probabilistici complessi.

Quando consideriamo più variabili casuali definite sullo stesso spazio di probabilità, possiamo introdurre la **distribuzione congiunta** [6]. Data una collezione di variabili $X, Y, \ldots$, la distribuzione congiunta fornisce la probabilità che ciascuna variabile assuma valori specifici o cada in determinati intervalli. Per due variabili si parla di distribuzione bivariata, mentre il concetto si estende a qualsiasi numero di variabili.

La distribuzione congiunta può essere rappresentata tramite:

- la funzione di distribuzione cumulativa congiunta,

- la funzione di densità congiunta (per variabili continue) o la funzione di massa congiunta (per variabili discrete).

Da questa è possibile derivare:

- le **distribuzioni marginali**, che descrivono ciascuna variabile indipendentemente dalle altre,
- le **distribuzioni condizionate**, che forniscono la probabilità di un sottoinsieme di variabili dato un particolare valore delle altre.

La **probabilità condizionata** misura la probabilità di un evento A assumendo che un altro evento B sia già verificato o noto [7]. Si indica generalmente con $P(A|B)$ e si definisce come:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0,$$

dove $P(A \cap B)$ rappresenta la probabilità che entrambi gli eventi si verifichino. La probabilità condizionata permette di aggiornare la conoscenza su un evento alla luce di informazioni aggiuntive.

Due eventi A e B si dicono **indipendenti** se il verificarsi di uno non influenza l'altro, ossia se $P(A|B) = P(A)$. In generale, $P(A|B)$ è distinto da $P(B|A)$, e confondere i due può portare a errori di ragionamento, come nel caso delle cosiddette *base rate fallacies*.

La probabilità condizionata può essere utilizzata in combinazione con il **teorema di Bayes** per invertire la relazione tra eventi:

$$P(A|B) = \frac{P(B|A)\,P(A)}{P(B)}.$$

Su questa base si sviluppa l'**inferenza bayesiana**, un approccio all'inferenza statistica che interpreta le probabilità non come frequenze o proporzioni osservate, bensì come livelli di fiducia (o gradi di credenza) nell'occorrenza di un evento. Il teorema di Bayes rappresenta dunque il fondamento formale di tale impostazione, consentendo di integrare conoscenza pregressa con nuove informazioni in modo coerente e quantitativo.

1.4 Modelli statistici e di apprendimento

Gran parte dei metodi per l'elaborazione e l'analisi dei dati testuali si fonda su strumenti della statistica e dell'apprendimento automatico. Questi modelli consentono di descrivere i dati come espressione di distribuzioni probabilistiche sottostanti, di individuare regolarità latenti e di valutare la capacità predittiva delle rappresentazioni ottenute. In questa sezione verranno presentati i concetti fondamentali alla base di tali approcci: dai modelli generativi, che descrivono il processo di produzione dei dati, alle tecniche di apprendimento non supervisionato per il raggruppamento documentale, fino agli strumenti dell'apprendimento statistico per affrontare problematiche come overfitting e underfitting. Infine, verranno introdotte misure formali quali la divergenza di Kullback–Leibler e la cross-entropy, centrali nella valutazione delle prestazioni dei modelli.

1.4.1 Modello generativo

Un **modello generativo** è un modello di apprendimento automatico progettato per apprendere la distribuzione dei dati di addestramento e generare nuovi campioni coerenti con tali dati. In altre parole, un modello generativo stima le distribuzioni di probabilità congiunte delle caratteristiche osservate nel dataset e utilizza queste informazioni per creare nuovi dati simili a quelli di addestramento [8].

Durante la fase di addestramento, i modelli generativi analizzano schemi e correlazioni presenti nei dati, spesso mediante tecniche di *learning* non supervisionate, in cui il modello riceve dati non etichettati e ne apprende la struttura interna. Il processo di addestramento prevede l'ottimizzazione di una **loss function**, che misura la discrepanza tra la distribuzione dei dati osservati e quella stimata dal modello. L'obiettivo è minimizzare questa discrepanza, in modo che i campioni generati rispecchino fedelmente le caratteristiche statistiche dei dati originali.

I modelli generativi trovano applicazione in numerosi ambiti, dalla generazione di testo e immagini alla sintesi audio, e costituiscono la base di molti approcci di intelligenza artificiale generativa recentemente sviluppati. La generazione di nuovi dati è intrinsecamente un processo probabilistico: il modello predice gli output più plausibili basandosi sulle distribuzioni apprese, senza possedere una comprensione

concettuale simile a quella umana. Questo approccio permette di simulare realisticamente fenomeni complessi e di creare contenuti nuovi coerenti con i pattern statistici dei dati di addestramento.

I modelli generativi mirano a creare nuovi dati coerenti con una determinata distribuzione, mentre i **modelli discriminativi** si concentrano sulla separazione di elementi in gruppi noti. In questo contesto, i **modelli di clustering** operano in maniera diversa: essi identificano autonomamente raggruppamenti all'interno di un insieme di dati, senza conoscenza preliminare del numero di cluster o della loro composizione.

1.4.2 Apprendimento non supervisionato e clustering documentale

L'**apprendimento non supervisionato** è un approccio del machine learning volto ad analizzare e raggruppare dati non etichettati, permettendo di scoprire strutture o pattern nascosti senza l'intervento umano [9]. Grazie a questa capacità, l'apprendimento non supervisionato è ampiamente impiegato per analisi esplorative dei dati, segmentazione dei clienti, strategie di cross-selling, riconoscimento di immagini e, più in generale, per l'identificazione di somiglianze e differenze nei dati.

Tra le principali tecniche di apprendimento non supervisionato, il **clustering** riveste un ruolo centrale. Il clustering consente di raggruppare elementi simili, evidenziando pattern e strutture nei dati grezzi. Per esempio, un ricercatore di mercato può utilizzare modelli di clustering per individuare segmenti di clienti o *buyer personas* all'interno di dati demografici, mentre in ambito documentale queste tecniche permettono di organizzare automaticamente collezioni di testi, evidenziando temi o argomenti ricorrenti.

Gli algoritmi di clustering possono essere classificati in diversi tipi: basati su densità, partitivi, gerarchici e probabilistici. In particolare, il **clustering probabilistico** si basa su modelli statistici nei quali i punti dati vengono raggruppati in funzione della probabilità di appartenenza a specifiche distribuzioni, permettendo così una gestione "soft" dei cluster.

1.4.3 Apprendimento statistico: overfitting, underfitting, annealing e early stopping

L'apprendimento non supervisionato e le tecniche di clustering rientrano nell'ambito più generale dell'**apprendimento statistico**, che si occupa di modellare e inferire pattern dai dati osservati [10]. In questo contesto, concetti come **overfitting** e **underfitting** risultano fondamentali per valutare la capacità di un modello di generalizzare correttamente, evitando di adattarsi eccessivamente ai dati di addestramento o, al contrario, di non cogliere le strutture rilevanti. Tecniche di ottimizzazione e regolarizzazione, invece, quali **annealing** e **early stopping**, vengono applicate per controllare il processo di apprendimento e migliorare la robustezza dei modelli.

Fondamenti di apprendimento statistico La teoria dell'apprendimento statistico costituisce il fondamento teorico dell'apprendimento automatico. Basandosi su strumenti derivati dalla statistica e dall'analisi funzionale, essa si propone di individuare una funzione capace di effettuare previsioni a partire dai dati osservati. Questo approccio ha portato a numerose applicazioni pratiche, tra cui la visione artificiale, il riconoscimento vocale, la bioinformatica e altre applicazioni che richiedono la modellizzazione di fenomeni complessi.

Gli obiettivi principali dell'apprendimento statistico consistono nella comprensione dei dati disponibili e nella previsione di dati futuri. I metodi si classificano in diverse categorie principali:

- **Apprendimento supervisionato**: è una tecnica che mira a istruire un sistema informatico affinché elabori automaticamente previsioni sui valori di uscita di un sistema rispetto a un input, basandosi su una serie di esempi ideali costituiti da coppie input-output fornite in fase di addestramento.

- **Apprendimento non supervisionato**: si distingue per il fatto che al sistema vengono forniti solo dati in input, senza etichette associate. Il compito del modello consiste nel riclassificare e organizzare i dati sulla base di caratteristiche comuni, individuando pattern latenti e relazioni tra gli esempi. A differenza dell'apprendimento supervisionato, le classi non sono note a priori

e devono essere apprese automaticamente dal sistema. Questa modalità è alla base di tecniche come il clustering e l'analisi delle componenti principali, utilizzate per scoprire strutture nascoste nei dati.

- **Apprendimento per rinforzo** (*reinforcement learning*): si focalizza sullo sviluppo di agenti autonomi capaci di scegliere azioni ottimali per raggiungere obiettivi specifici interagendo con un ambiente. A differenza dei paradigmi supervisionato e non supervisionato, questo approccio affronta problemi di decisioni sequenziali, in cui ogni azione influisce sullo stato futuro del sistema.

  La qualità di ciascuna azione è valutata tramite un segnale numerico di *ricompensa*, che guida l'agente verso comportamenti corretti. I problemi di reinforcement learning sono solitamente modellizzati tramite i processi decisionali di Markov e possono essere risolti attraverso algoritmi differenti, classificabili secondo l'utilizzo di un modello dell'ambiente, le modalità di raccolta dell'esperienza (diretta o indiretta) e la rappresentazione degli stati e delle azioni (discreta o continua).

Overfitting e underfitting La qualità di una funzione stimata f dipende dalla sua capacità di generalizzare su dati mai osservati. Due problemi fondamentali emergono in questo contesto:

- **Overfitting**: il modello si adatta eccessivamente al training set, includendo rumori o dettagli irrilevanti, compromettendo la generalizzazione. Produce performance elevate sui dati di addestramento ma scarse su dati nuovi.

- **Underfitting**: il modello è troppo semplice per cogliere le relazioni sottostanti nei dati, portando a errori elevati sia sui dati osservati sia su quelli futuri.

Annealing ed early stopping Per mitigare l'overfitting e l'underfitting, si adottano tecniche di ottimizzazione e regolarizzazione durante l'addestramento:

- **Early stopping**: interrompe l'addestramento prima che il modello inizi a sovradattarsi, monitorando le performance su un set di validazione.

- **Annealing** (o *learning rate annealing*): riduce progressivamente il tasso di apprendimento durante l'ottimizzazione, stabilizzando il modello vicino a minimi globali della funzione di perdita e migliorando la generalizzazione.

Queste strategie risultano particolarmente utili nei contesti supervisionati con dati complessi, dove bilanciare capacità del modello e generalizzazione è cruciale per ottenere previsioni affidabili.

1.4.4 Divergenza di Kullback–Leibler e cross-entropy

L'analisi di fenomeni come **overfitting** e **underfitting**, insieme a tecniche di ottimizzazione come **annealing** e **early stopping**, evidenzia l'importanza di metriche quantitative per valutare quanto le distribuzioni predette da un modello si discostino dai dati reali. In questo contesto, la **divergenza di Kullback–Leibler** e la **cross-entropy** rappresentano strumenti fondamentali per misurare la distanza tra distribuzioni probabilistiche, guidando l'ottimizzazione dei modelli e la prevenzione dell'overfitting.

Divergenza di Kullback–Leibler In teoria della probabilità e in teoria dell'informazione, la divergenza di Kullback–Leibler (KL) misura la differenza tra due distribuzioni di probabilità P e Q. Indicata con $D_{KL}(P\|Q)$, rappresenta l'informazione persa quando Q approssima P:

$$D_{KL}(P\|Q) = \sum_i P(i) \log_2 \frac{P(i)}{Q(i)}.$$

Cross-Entropy Nei problemi di classificazione, la valutazione della performance di un modello non si limita a verificare se la predizione sia corretta, ma richiede anche di quantificare quanto la stima del modello si discosti dalla realtà. A questo scopo, le *loss function* forniscono un'indicazione quantitativa dell'errore commesso dal modello, guidandone l'ottimizzazione.

La Cross-Entropy Loss svolge un ruolo cruciale in questo contesto, in quanto penalizza in misura maggiore le predizioni sbagliate effettuate con alta confidenza. In altre parole, se un modello assegna una probabilità elevata a una classe errata, la cross-entropy assegna un errore più consistente rispetto a una predizione incerta, incentivando il modello a correggere con maggiore attenzione le proprie stime.

Questo meccanismo rende la cross-entropy particolarmente efficace nei compiti di classificazione, sia binaria (ad esempio distinguere email spam da email lecite) sia multiclasse (ad esempio classificare immagini tra diverse categorie come gatti, cani o uccelli), assicurando che il modello apprenda in modo sensibile agli errori più gravi.

La cross-entropy può essere formalmente espressa anche in termini della divergenza di Kullback–Leibler. In particolare, la cross-entropy di q rispetto a p può essere scritta come:

$$H(p, q) = H(p) + D_{KL}(p\|q),$$

dove $H(p)$ rappresenta l'entropia della distribuzione vera p.

Questa relazione chiarisce come la minimizzazione della cross-entropy durante l'addestramento di un modello equivalga a ridurre la distanza tra la distribuzione predetta dal modello e quella reale, guidando l'ottimizzazione del modello verso predizioni più accurate e una migliore capacità di generalizzazione.

Capitolo 2

Probabilistic Latent Semantic Analysis

2.1 Latent Semantic Analysis

L'*Analisi Semantica Latente* (LSA) è una variante avanzata del VSM. Si basa sull'utilizzo della SVD per ridurre la dimensionalità dello spazio termine-documento e affrontare i problemi di polisemia e sinonimia. L'idea centrale del LSA è la seguente: i termini che compongono un documento suggeriscono il contenuto semantico del documento stesso. Tuttavia, ogni singolo termine è un indicatore "non perfetto" di concetti latenti. La variabilità nell'uso delle parole nasconde parzialmente la vera struttura semantica del documento. Riducendo la dimensionalità dello spazio termine-documento, si riesce a identificare automaticamente le relazioni semantiche tra documenti. Si elimina il cosiddetto rumore lessicale, ovvero le differenze nell'uso delle parole, tra cui sinonimi e ripetizioni.

Può essere applicata a dati bimodali, ovvero dati di conteggio su un dominio discreto binario. Tuttavia, poiché la principale applicazione di LSA riguarda l'analisi e il recupero di documenti testuali, in questo lavoro ci concentreremo su quest'ultima tipologia di dato.

Si supponga quindi di disporre di una collezione di documenti testuali $\mathcal{D} = \{d_1, \ldots, d_N\}$ con termini estratti da un vocabolario $\mathcal{W} = \{w_1, \ldots, w_M\}$. Ignorando l'ordine sequenziale in cui le parole compaiono all'interno di ciascun documento, i

dati possono essere riassunti in una tabella di co-occorrenza

$$A = \big(n(d_i, w_j)\big)_{i,j}$$

di dimensione $N \times M$, dove $n(d_i, w_j)$ indica il numero di occorrenze del termine w_j nel documento d_i.

In questo caso particolare, A è nota anche come matrice termine-documento e le sue righe e colonne sono rispettivamente dette vettori termine e vettori documento. L'assunzione di base è che la rappresentazione semplificata dei documenti, ad esempio tramite il modello **"Bag-of-Words"** o una codifica vettoriale, conservi in molti casi gran parte delle informazioni rilevanti, come nel caso di compiti di recupero testuale basato su parole chiave.

La rappresentazione tramite tabella di co-occorrenza mette subito in evidenza il problema della scarsità dei dati, noto anche come *problema delle frequenze nulle*. Una tipica matrice termine-documento ottenuta da articoli brevi, riassunti o abstract può contenere solo una piccola frazione di valori non nulli (solitamente inferiore all'1%), riflettendo il fatto che solo pochi termini del vocabolario vengono effettivamente utilizzati in ciascun documento.

Questa condizione ha implicazioni dirette per le applicazioni che si basano sul confronto tra query e documenti e sulla valutazione della similarità tra documenti in base ai termini condivisi. Infatti, la probabilità di trovare numerosi termini comuni anche tra articoli strettamente correlati è spesso bassa, proprio perché potrebbero non utilizzare esattamente gli stessi termini.

Ad esempio, molte funzioni di matching usate in questo ambito si basano su misure di similarità fondate sul prodotto interno tra coppie di vettori documento. Tuttavia, emergono due problemi principali: da un lato, è necessario considerare i **sinonimi** per non sottostimare la vera similarità tra documenti; dall'altro, è necessario gestire la **polisemia** (polysemy) per evitare di sovrastimare la similarità contando termini comuni usati con significati diversi. Entrambi questi aspetti possono portare a punteggi di similarità lessicale non adeguati, che non riflettono la "vera" similarità nascosta nella semantica delle parole.

2.1.1 Analisi Semantica Latente tramite Decomposizione ai Valori Singolari

L'idea chiave di LSA consiste nel mappare documenti e, per simmetria, anche termini, in uno spazio vettoriale a dimensionalità ridotta, chiamato **spazio semantico latente**. Nelle applicazioni tipiche di indicizzazione, i documenti hanno una dimensionalità dell'ordine di circa 100–300.

La trasformazione dei vettori documento e termine nello spazio latente avviene tramite una proiezione lineare, ottenuta decomponendo la matrice di co-occorrenza attraverso la Decomposizione ai Valori Singolari (Singular Value Decomposition). In particolare, si parte dalla SVD della matrice $A \in \mathbb{R}^{N \times M}$, cioè

$$A = U \Sigma V^\top,$$

dove $U \in \mathbb{R}^{N \times N}$ e $V \in \mathbb{R}^{M \times M}$ sono matrici ortogonali, ovvero $U^\top U = I_N$ e $V^\top V = I_M$, mentre $\Sigma \in \mathbb{R}^{N \times M}$ è diagonale a blocchi e contiene i valori singolari di A.

La matrice A viene approssimata mediante una versione a rango ridotto, ottenuta troncando la sua SVD e annullando tutti i valori singolari a eccezione dei K più grandi. Il risultato è la matrice:

$$\tilde{A} = U \tilde{\Sigma} V^\top \approx A,$$

che rappresenta l'approssimazione ottimale di rango K ($< min(N, M)$) della matrice originale A, in termini della norma di Frobenius (o norma L_2). Tale proprietà è garantita dal noto teorema di Eckart–Young [11].

Se vogliamo calcolare i prodotti interni documento-documento sulla base di questa approssimazione, otteniamo

$$\tilde{A}\tilde{A}^\top = U \tilde{\Sigma}^2 U^\top,$$

il che suggerisce di considerare le righe di $U\tilde{\Sigma}$ come coordinate dei documenti nello spazio latente.

Mentre i vettori originali sono tipicamente sparsi, i corrispondenti vettori latenti non lo sono, il che consente di calcolare valori di associazione significativi tra coppie di documenti anche quando questi non condividono termini in comune. L'ipotesi

è che termini con significato simile vengano approssimativamente trasformati nella stessa direzione nello spazio latente.

Sebbene la LSA abbia ottenuto successi significativi in diversi domini, inclusa l'indicizzazione automatica (Latent Semantic Indexing), essa presenta alcune limitazioni dovute alla sua debole fondazione statistica. Per ovviare a questo problema è stata introdotta la Probabilistic Latent Semantic Analysis (PLSA), che si basa sul principio di verosimiglianza e definisce un modello generativo adeguato per i dati. Ciò consente di applicare tecniche statistiche standard per la stima del modello, la combinazione di modelli e il controllo della complessità, oltre a gestire in modo esplicito la polisemia, distinguendo i diversi significati e usi delle parole.

2.2 Analisi Probabilistica Semantica Latente

L'**Analisi Probabilistica Semantica Latente** (*Probabilistic Latent Semantic Analysis*), introdotta da Thomas Hofmann nel 1999 [12], è una tecnica statistica per analizzare strutture di dati che rappresentano la co-occorrenza tra due insiemi distinti di elementi, come parole e documenti, che trova applicazione nel recupero e filtraggio delle informazioni, nell'elaborazione del linguaggio naturale, nell'apprendimento automatico da testo e in settori affini. Il modello viene adattato a partire da un corpus di addestramento composto da documenti testuali, attraverso una generalizzazione dell'algoritmo di **Expectation-Maximization** (EM). Questo approccio comporta almeno due vantaggi significativi:

- esso supera il problema della polisemia, ovvero la presenza di più significati per una stessa parola, riuscendo a distinguere il senso corretto in base al contesto d'uso;

- esso è in grado di individuare somiglianze tematiche tra parole, raggruppando termini che ricorrono in contesti simili. In particolare, ciò consente di riconoscere casi di sinonimia, cioè parole diverse che condividono un significato uguale o molto vicino.

Rispetto all'Analisi Semantica Latente classica, che si basa sull'algebra lineare ed esegue una decomposizione ai valori singolari (SVD) delle tabelle di co-occorrenza, il metodo proposto si fonda su una decomposizione a mescolanza derivata da un modello a classi latenti.

2.2.1 Aspect Model

Il punto di partenza per la *Probabilistic Latent Semantic Analysis*, da ora in poi abbreviata con *PLSA*, è un modello statistico noto come **Aspect Model** [13]. Si tratta di un *modello a variabili latenti* per dati di co-occorrenza, in cui a ogni occorrenza di una parola $w \in W = \{w_1, \ldots, w_M\}$ in un documento $d \in D = \{d_1, \ldots, d_N\}$ viene associata una *variabile latente non osservabile* $z \in Z = \{z_1, \ldots, z_K\}$.

Si introducono le seguenti probabilità:

- $P(d)$: probabilità che un'occorrenza di parola si verifichi nel documento d,
- $P(w|z)$: probabilità condizionata di una parola w dato lo stato latente z,
- $P(z|d)$: distribuzione di probabilità, specifica per ogni documento, sulla variabile latente.

Utilizzando queste definizioni, si può descrivere il **modello generativo** per le co-occorrenze parola/documento attraverso il seguente schema:

- selezionare un documento d con probabilità $P(d)$;
- scegliere una classe latente z con probabilità $P(z|d)$;
- generare una parola w con probabilità $P(w|z)$.

Come risultato si ottiene una coppia osservata (d, w), mentre la variabile latente z viene scartata.

Traducendo questo processo in un **modello di probabilità congiunta**, si ottiene l'espressione:

$$P(d, w) = P(d)\, P(w|d) \tag{2.1}$$

dove

$$P(w|d) = \sum_{z \in Z} P(w|z)\, P(z|d) \tag{2.2}$$

Fondamentalmente, per derivare (2.2) si deve sommare sulle possibili scelte di Z che potrebbero aver generato l'osservazione.

Aspect Model è uno *statistical mixture model* che si fonda su un'ipotesi di indipendenza condizionata: si assume che il documento d e la parola w siano indipendenti dallo stato della variabile latente associata.

Poiché la cardinalità di z è inferiore al numero di documenti o parole nella collezione, z agisce come una variabile collo di bottiglia nel processo di predizione delle parole, limitando la complessità del modello e forzando una rappresentazione compatta delle co-occorrenze.

A differenza dei *modelli di clustering documentale*, le distribuzioni dei documenti $P(w|d)$ sono ottenute come combinazioni convesse degli aspetti $P(w|z)$. I documenti non vengono assegnati a cluster, ma sono caratterizzati da una specifica mescolanza di fattori, con pesi $P(z|d)$. Questi **pesi di mescolanza** offrono maggiore potere modellistico e sono diversi dalle probabilità a posteriori nei modelli di clustering.

Seguendo il principio di massima verosimiglianza, si determinano $P(d)$, $P(z|d)$ e $P(w|z)$ massimizzando la funzione **likelihood** (*log-verosimiglianza*):

$$\mathcal{L}(\{n(d,w)\}_{d \in D, w \in W}) = \sum_{d \in D} \sum_{w \in W} n(d,w) \log P(d,w) \tag{2.3}$$

dove $n(d,w)$ rappresenta la frequenza del termine, cioè il numero di volte in cui w appare in d.

È interessante osservare che il modello può essere parametrizzato in modo equivalente (Figura 2.1) tramite la seguente formulazione simmetrica:

$$P(d,w) = \sum_{z \in Z} P(z)\, P(d|z)\, P(w|z) \tag{2.4}$$

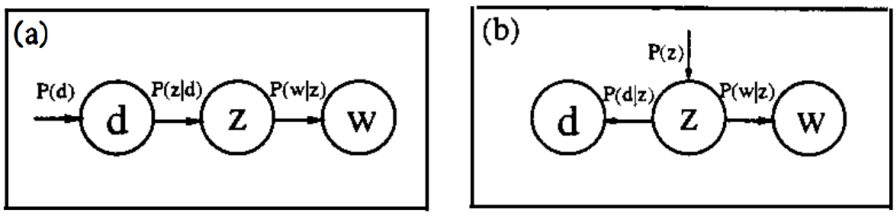


Figura 2.1: Rappresentazione grafica dell'Aspect Model nella parametrizzazione asimmetrica (a) e simmetrica (b) (figura tratta da [14])

Tale formulazione è *perfettamente simmetrica* rispetto alle due entità considerate: documenti e parole.

Confronto tra Aspect Model e modelli statistici di clustering

Nei modelli di clustering per documenti, solitamente si associa una variabile di classe latente a ciascun documento della collezione.

Si dimostra che la probabilità condizionata di una parola in un modello probabilistico di clustering è data da:

$$P(w_j|d_i) = \sum_{k=1}^{K} P(\{c(d_i)) = c_k\} P(w_j|c_k) \tag{2.5}$$

dove $P(\{c(d_i) = c_k\})$ è la probabilità *a posteriori* che il documento d_i appartenga al cluster c_k.

In un modello di clustering per documenti si assume che ogni documento appartenga esattamente a un solo cluster e che solo la finitezza del numero di osservazioni per documento induca incertezza sull'appartenenza del documento al cluster, come espresso dalle probabilità a posteriori $P\{c(d_i) = c_k\}$.

Al contrario, Aspect Model assume che ogni occorrenza di una parola in un documento sia associata a uno stato z_k della variabile della classe latente. Questo non esclude che occorrenze diverse di parole nello stesso documento, o occorrenze della stessa parola in documenti diversi, possano essere spiegate da aspetti differenti. Tuttavia, poiché le variabili di classe latente associate alle occorrenze nello stesso documento condividono le stesse proprietà *a priori* $P(z_k|d_i)$, le osservazioni all'interno di un documento risultano accoppiate.

Come risultato di questo accoppiamento, le probabilità $P(z_k|d_i)$ e $P(z_k|w_j)$ tendono a essere sparse, cioè, per un dato d_i o w_j, solitamente solo poche di esse sono significativamente diverse da zero.

L'Aspect Model effettua il clustering delle coppie documento–parola, che è diverso dal fare clustering dei soli documenti, delle sole parole o di entrambi separatamente. Di conseguenza, le probabilità specifiche di parola in un documento nell'Aspect Model sono una combinazione convessa di aspetti, mentre il modello di clustering assume che vi sia un'unica distribuzione specifica del cluster, condivisa da tutti i documenti appartenenti a quel cluster.

Applicazione dell'Aspect Model

È stato considerato un dataset (CLUSTER) contenente gli abstract di 1568 documenti sul clustering ed è stato addestrato un Aspect Model con 128 classi latenti (è possibile trovarli in [13]). Come termine "interessante" in questo dominio è stata scelta la parola *segment*.

Aspect 1	Aspect 2	Aspect 3	Aspect 4
imag	video	region	speaker
SEGMENT	sequenc	contour	speech
textur	motion	boundari	recogni
color	frame	descrip	signal
tissu	scene	imag	train
brain	SEGMENT	SEGMENT	hmm
slice	shot	precis	sourc
cluster	imag	estim	speakerindepend
mri	cluster	pixel	SEGMENT
algorithm	visual	paramet	sound

Figura 2.2: I 4 aspetti con la più alta probabilità di generare la parola «segment», derivati da un Aspect Model con $K = 128$ sulla raccolta di documenti CLUSTER. Le radici delle parole mostrate sono le parole più probabili nella distribuzione condizionata alla classe $P(w_j|z_k)$, elencate dall'alto verso il basso in ordine decrescente di probabilità [13]

La tabella riportata in Figura 2.2 mostra le parole più probabili appartenenti a 4 dei 128 aspetti che presentano la probabilità più alta di generare il termine "segment". Questa caratterizzazione sintetica degli aspetti rivela sotto-domini molto significativi: i primi tre aspetti riguardano la segmentazione di immagini, mentre il quarto si riferisce alla segmentazione fonetica nel contesto del riconoscimento vocale. La suddivisione ulteriore della segmentazione di immagini in tre tipi leggermente

diversi di uso della parola "segment" è anch'essa molto plausibile: il primo aspetto sembra catturare alcune statistiche di termini nell'imaging medico, il secondo riguarda l'analisi di sequenze di immagini, mentre il terzo è legato principalmente alla segmentazione di immagini nel contesto del rilevamento di contorni e confini. Si noti che i termini "region" e "segment" hanno un significato molto simile in questo contesto, come effettivamente riflesso da questo aspetto.

2.2.2 Addestramento del modello con l'algoritmo EM

La procedura standard per la stima della massima verosimiglianza è l'**algoritmo di Expectation-Maximization** (*EM*). Tale algoritmo alterna due passaggi:

1. uno step di Expectation (E), in cui si calcolano le probabilità a posteriori delle variabili latenti;

2. uno step di Maximization (M), in cui i parametri vengono aggiornati.

Il passo **E (Expectation)** dell'algoritmo EM si basa sull'applicazione della *formula di Bayes*, che consente di calcolare la probabilità che una determinata parola w, osservata in un documento d, sia associata a una particolare classe latente z:

$$P(z|d,w) = \frac{P(w|z)P(z|d)}{\sum_{z' \in Z} P(w|z')P(z'|d)} \tag{2.6}$$

In altre parole, questa quantità rappresenta la probabilità *a posteriori* che la classe latente z abbia generato la coppia osservata (d, w).

Il successivo passo **M (Maximization)** aggiorna i parametri del modello sulla base delle probabilità calcolate nel passo E. L'obiettivo è massimizzare il logaritmo dell'attesa della verosimiglianza completa:

$$\mathbb{E}[\mathcal{L}^c] = \sum_{i=1}^{N} \sum_{j=1}^{M} n(d_i, w_j) \sum_{k=1}^{K} P(z_k|d_i, w_j) \log \left[P(w_j|z_k)P(z_k|d_i)\right] \tag{2.7}$$

Per tenere conto dei vincoli di normalizzazione, l'equazione (2.7) deve essere estesa con opportuni moltiplicatori di Lagrange τ_k e ρ_i:

$$\mathcal{H} = \mathbb{E}\left[\mathcal{L}^c\right] + \sum_{k=1}^{K} \tau_k \left(1 - \sum_{j=1}^{M} P(w_j|z_k)\right) + \sum_{i=1}^{N} \rho_i \left(1 - \sum_{k=1}^{K} P(z_k|d_i)\right) \tag{2.8}$$

La massimizzazione di $\mathcal{H}$ rispetto alle funzioni di massa di probabilità porta al seguente insieme di equazioni stazionarie:

$$\sum_{i=1}^{N} n(d_i, w_j) P(z_k|d_i, w_j) - \tau_k \, P(w_j|z_k) = 0, \quad 1 \leq j \leq M, \; 1 \leq k \leq K, \tag{2.9}$$

$$\sum_{j=1}^{M} n(d_i, w_j) P(z_k|d_i, w_j) - \rho_i \, P(z_k|d_i) = 0, \quad 1 \leq i \leq N, \; 1 \leq k \leq K. \tag{2.10}$$

Si arriva così alle seguenti **equazioni di ri-stima**, che aggiornano i parametri del modello:

$$P(w_j|z_k) = \frac{\sum_{i=1}^{N} n(d_i, w_j) \, P(z_k|d_i, w_j)}{\sum_{m=1}^{M} \sum_{i=1}^{N} n(d_i, w_m) \, P(z_k|d_i, w_m)} \tag{2.11}$$

$$P(z_k|d_i) = \frac{\sum_{j=1}^{M} n(d_i, w_j) \, P(z_k|d_i, w_j)}{n(d_i)} \tag{2.12}$$

dove la quantità $n(d_i) = \sum_{j=1}^{M} n(d_i, w_j)$ rappresenta la lunghezza (cioè il numero totale di occorrenze di parole) del documento d_i.

2.2.3 Ottimizzazione del modello e generalizzazione: il Tempered EM

Con l'algoritmo EM ci siamo concentrati sulla stima di massima verosimiglianza per adattare il modello a una collezione di documenti. Tuttavia, è fondamentale distinguere tra la qualità del modello sui dati di addestramento e la sua capacità di generalizzare su dati non visti. Per misurare questa capacità, introduciamo la *perplexity*, una metrica comunemente usata nel *language modeling* e che valuta

l'inverso della probabilità media logaritmica sui dati di test:

$$\mathcal{P}(\{n'(d_i, w_j)\}_{i,j}) = \exp\left(-\frac{\sum_{i,j} n'(d_i, w_j) \log P(w_j|d_i)}{\sum_{i,j} n'(d_i, w_j)}\right) \tag{2.13}$$

dove $n'(d_i, w_j)$ indica la frequenza osservata della coppia (d_i, w_j) nei dati di test.

Per affrontare il problema fondamentale della generalizzazione su dati non osservati — centrale nella teoria dell'apprendimento statistico — è stata proposta una generalizzazione della stima di massima verosimiglianza, nota come *annealing*. Questo approccio conduce al metodo *Tempered Expectation-Maximization* (TEM).

L'idea alla base del TEM è formulare l'E-step come problema di ottimizzazione, che, nei modelli a variabili latenti, equivale a minimizzare una funzione obiettivo, detta *energia libera di Helmholtz*, che per l'Aspect Model è:

$$\begin{aligned}\mathcal{F}_\beta = &- \beta \sum_{i=1}^{N} \sum_{j=1}^{M} n(d_i, w_j) \sum_{k=1}^{K} \tilde{P}(z_k; d_i, w_j) \log \left[P(d_i|z_k)P(w_j|z_k)P(z_k)\right] \\ &+ \sum_{i=1}^{N} \sum_{j=1}^{M} n(d_i, w_j) \sum_{k=1}^{K} \tilde{P}(z_k; d_i, w_j) \log \tilde{P}(z_k; d_i, w_j),\end{aligned} \tag{2.14}$$

dove:

- $\tilde{P}(z; d, w)$ rappresenta una distribuzione variazionale (cioè parametrica) sulla variabile latente z;
- β è un parametro chiamato *inverse computational temperature*, in analogia con la fisica statistica.

Il primo termine rappresenta la log-verosimiglianza attesa (negativa), scalata per β, mentre il secondo è l'entropia delle distribuzioni. Quando $\beta = 1$ e $\tilde{P}(z_k; d_i, w_j) = P(z_k|d_i, w_j)$, si recupera esattamente l'E-step standard dell'EM.

In generale, il valore ottimale di $\tilde{P}$ è dato da:

$$\tilde{P}(z_k; d_i, w_j) = \frac{\left[P(z_k)P(d_i|z_k)P(w_j|z_k)\right]^\beta}{\sum_{l=1}^{K} \left[P(z_l)P(d_i|z_l)P(w_j|z_l)\right]^\beta} = \frac{\left[P(z_k|d_i)P(w_j|z_k)\right]^\beta}{\sum_{l=1}^{K} \left[P(z_l|d_i)P(w_j|z_l)\right]^\beta} \tag{2.15}$$

Il parametro β regola l'influenza dell'entropia: valori di $\beta < 1$ rendono le distribuzioni posteriori più uniformi (meno concentrate), favorendo la generalizzazione e riducendo l'overfitting.

Contrariamente allo schema di *annealing* standard (che parte da $\beta \ll 1$ e lo aumenta), si propone qui una strategia inversa: si inizia con $\beta = 1$ e si diminuisce progressivamente β finché le prestazioni sul set di validazione non peggiorano. Questo approccio "inverso" può accelerare significativamente l'adattamento del modello (anche di un fattore tra 10 e 50), senza compromettere le prestazioni sui dati di test, che risultano comparabili a quelle ottenute con un'annealing più attento e graduale.

Algoritmo TEM

L'algoritmo TEM può essere riassunto nei seguenti passi:

1. Si imposta $\beta \leftarrow 1$ e si esegue l'algoritmo EM con *early stopping*.

2. Si aggiorna $\beta \leftarrow \eta \cdot \beta$, con $\eta < 1$, e si esegue un'iterazione di TEM.

3. Se le prestazioni sui dati di validazione continuano a migliorare in modo significativo, si proseguono le iterazioni con il nuovo valore di β; altrimenti si torna al punto 2.

4. Il procedimento viene interrotto quando ulteriori riduzioni di β non comportano miglioramenti apprezzabili.

2.2.4 Geometria dell'Aspect Model e Spazio Semantico Latente Probabilistico

Consideriamo le distribuzioni multinomiali condizionate alla classe $P(\cdot|z_k)$ sul vocabolario W, note nel contesto dell'Aspect Model come *fattori*. Ciascuna di queste distribuzioni può essere rappresentata come un punto nel simplesso di dimensione $M - 1$, che racchiude tutte le possibili multinomiali definite su W.

Indichiamo con $\text{conv}(\cdot)$ l'**inviluppo convesso** (convex hull) di un insieme di punti, cioè l'insieme di tutte le combinazioni lineari con coefficienti non negativi

che sommano a 1:

$$\mathrm{conv}(\{x_1, \ldots, x_K\}) = \Big\{ \sum_{i=1}^{K} \lambda_i x_i \;\Big|\; \lambda_i \geq 0, \sum_{i=1}^{K} \lambda_i = 1 \Big\}.$$

L'inviluppo convesso di questi K punti genera quindi una regione convessa R di dimensione $K-1$:

$$R \equiv \mathrm{conv}\big(P(\cdot|z_1), \ldots, P(\cdot|z_K)\big),$$

definita sul simplesso, a condizione che i punti siano in posizione generale.

L'ipotesi di modellizzazione espressa dall'equazione (2.1) assume che, per ogni documento d_i con $1 \leq i \leq N$, la distribuzione di probabilità condizionata $P(\cdot|d_i)$ possa essere approssimata come una combinazione convessa di K funzioni di massa di probabilità $P(\cdot|z_k)$. I coefficienti di combinazione, ovvero le probabilità $P(z_k|d_i)$, rappresentano le coordinate che identificano univocamente la posizione di ciascun documento all'interno della regione convessa R.

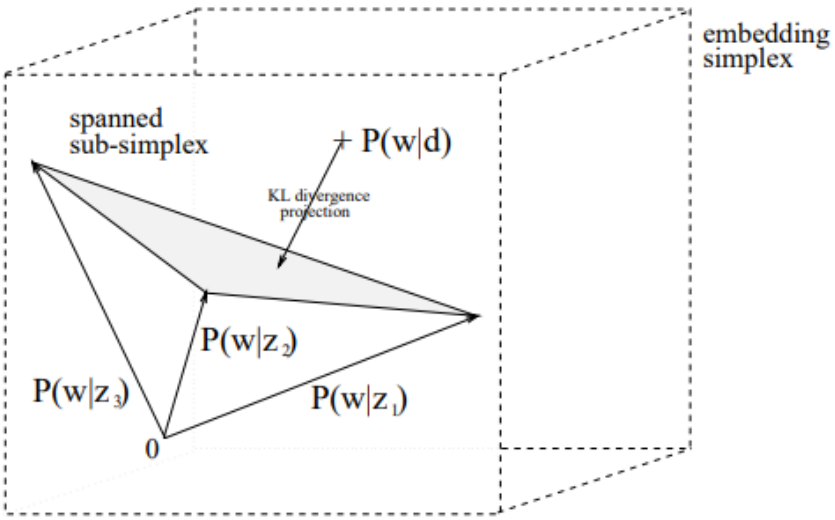


Figura 2.3: Simplesso delle probabilità e di una regione convessa generate dalle probabilità condizionate alla classe nell'Aspect Model [12]

Dal punto di vista geometrico (Figura 2.3), ciò implica che, pur partendo da variabili latenti discrete, il modello induce uno spazio latente continuo immerso nello spazio di tutte le funzioni di massa di probabilità definite sul vocabolario W.

Poiché la dimensionalità della regione convessa R è $K-1$ anziché $M-1$ come per il simplesso delle probabilità, ciò può essere interpretato anche come una riduzione di dimensionalità ed R può essere identificato come lo **spazio semantico latente probabilistico**.

Poiché l'Aspect Model è simmetrico rispetto a termini e documenti, invertendo i loro ruoli si ottiene una regione corrispondente R_0 nel simplesso di tutte le funzioni di massa di probabilità su D.

2.3 Una visione moderna del LSA

Negli anni successivi alla sua formulazione originale, il *Probabilistic Latent Semantic Analysis* (PLSA) è stato reinterpretato e riformulato in modi che ne rendono più chiara l'intuizione e il collegamento con altri modelli generativi di argomenti, come la *Latent Dirichlet Allocation* (LDA). Questa "visione moderna" non modifica le fondamenta matematiche del PLSA, ma riorganizza le sue ipotesi di base in un linguaggio più vicino alle attuali metodologie di modellazione probabilistica [15].

L'idea è partire dalla descrizione del *processo generativo* che il PLSA sottintende, rendendo esplicite le assunzioni implicite nella formulazione originale:

- cosa rappresentano i parametri ϕ e θ;
- come avviene la generazione di un documento e dei termini che lo compongono;
- qual è il ruolo delle variabili latenti e come vengono gestite nell'algoritmo EM.

A partire da queste ipotesi di base, il modello può essere reinterpretato in una forma più chiara, che agevola sia l'analisi concettuale sia l'estensione verso modelli più complessi.

Introduciamo preliminarmente alcune ipotesi: per prima cosa, si assume che un argomento ϕ_k sia una distribuzione su un vocabolario V di dimensione fissa. Nel modello PLSA originale, questa distribuzione non è specificata in modo esplicito, ma è di tipo multinomiale. Pertanto, ϕ_k è essenzialmente un vettore in cui ogni elemento $\phi_{(k,w)}$ rappresenta la probabilità che il termine w sia scelto dall'argomento k, ossia:

$$P(w|k) = \phi_{(k,w)} \tag{2.16}$$

con $\sum_w^V \phi_{(k,w)} = 1$.

In secondo luogo, si assume che un documento sia composto da più argomenti. Perciò, per ciascun documento d, esiste una distribuzione θ_d su un numero fisso di argomenti T. Anche in questo caso, nel PLSA originale non vi è una specifica esplicita, ma la distribuzione è multinomiale, e ogni elemento $\theta_{(d,k)}$ rappresenta la probabilità che l'argomento k compaia nel documento d:

$$P(k|d) = \theta_{(d,k)} \tag{2.17}$$

con $\sum_k^T \theta_{(d,k)} = 1$.

Questo costituisce la premessa del modello.

Prima di procedere, occorre evidenziare una sottile distinzione: la differenza tra un termine w nel vocabolario V e una posizione di token d_i in un documento d. I termini nel vocabolario sono distinti, mentre le posizioni di token rappresentano gli slot in cui i termini vengono collocati. Di conseguenza, uno stesso termine può comparire più volte in un documento, in posizioni diverse.

Immaginando il processo di scrittura di un documento:

- per ogni posizione i, si sceglie dapprima un argomento in base alla distribuzione θ_d;
- scelto l'argomento k, si seleziona un termine in base alla distribuzione ϕ_k.

Questo processo a due fasi si ripete per tutte le posizioni di token e per tutti i documenti del dataset.

Il processo generativo può essere riassunto come segue:

- Per ogni documento d:
  - Per ogni posizione di token i:
    - Scegli un topic $z \sim$ Multinomiale(θ_d)

* Scegli un termine $w \sim$ Multinomiale(ϕ_z)

La probabilità che un termine w appaia alla posizione i del documento d è:

$$P(d_i = w|\Phi, \theta_d) = \sum_{z=k}^{T} \phi_{(z,w)}\, \theta_{(d,z)} \tag{2.18}$$

La probabilità congiunta dell'intero dataset W è:

$$P(\mathcal{W}|\Phi, \Theta) = \prod_{d}^{D} \prod_{i}^{N_d} \sum_{z=k}^{T} \phi_{(z,w)} \theta_{(d,z)} = \prod_{d}^{D} \prod_{w}^{V} \left(\sum_{z=k}^{T} \phi_{(z,w)}\, \theta_{(d,z)} \right)^{n(d,w)} \tag{2.19}$$

dove $n(d, w)$ è il numero di volte in cui il termine w compare nel documento d.

La verosimiglianza dipende dai parametri Φ e Θ, che devono essere stimati dai dati. Si vogliono ottenere parametri che possano massimizzare la verosimiglianza. Pertanto:

$$\arg\max_{\Phi,\Theta} \left[\log P(\mathcal{W}|\Phi, \Theta) + \sum_{d}^{D} \lambda_d \left(1 - \sum_{z}^{T} \theta_{(d,z)} \right) + \sum_{z}^{T} \sigma_z \left(1 - \sum_{w}^{V} \phi_{(z,w)} \right) \right] \tag{2.20}$$

dove i termini con λ_d e σ_z sono moltiplicatori di Lagrange per garantire che le distribuzioni multinomiali siano normalizzate, ovvero comprese nell'intervallo [0,1].

La massimizzazione diretta è difficile poiché il logaritmo si trova all'esterno di una sommatoria. Per questo, si utilizza l'algoritmo EM.

L'assunzione chiave per applicare EM è supporre di conoscere, per ogni posizione di token, quale argomento sia stato scelto. In altre parole, per ogni posizione di token si conosce il valore z. Naturalmente, si tratta di un'informazione ipotetica. Si indica con $R_{w_{d_i}}$ la variabile che rappresenta quale argomento z è stato scelto per la posizione i nel documento d per il termine w ed è un vettore di dimensione T, con $\sum_k R_{(w_{d_i},k)} = 1$. Questo implica che ogni $R_{w_{d_i}}$ è di fatto una distribuzione valida e che $\mathbf{R}$ è una matrice in cui ogni riga corrisponde a uno di tali vettori.

Sostituendo queste variabili latenti nella funzione di verosimiglianza, si ottiene:

$$\mathcal{L} = \log P(W|\mathbf{R}, \Phi, \Theta) = \sum_{d}^{D} \sum_{d_i}^{N_d} \sum_{z}^{T} R_{(w_{d_i},z)} \left(\log \phi_{(z,w_{d_i})} + log\theta_{(d,z)} \right) \tag{2.21}$$

Il nuovo obiettivo di ottimizzazione diventa:

$$\arg\max_{\Phi,\Theta} \Lambda = \log P(W|\mathbf{R}, \Phi, \Theta) + \sum_{d}^{D} \lambda_d \left(1 - \sum_{z}^{T} \theta_{(d,z)} \right) + \sum_{z}^{T} \sigma_k \left(1 - \sum_{w}^{V} \phi(z,w) \right) \tag{2.22}$$

- **E-step**
  Si calcola la distribuzione a posteriori delle variabili latenti, dato il dataset e i valori correnti dei parametri:

$$< R_{(w_{d_i},k)} >= \frac{\phi_{(k,w_{d_i})}\theta_{(d,k)}}{\sum_k^T \phi_{(k,w_{d_i})}\theta_{(d,k)}} \tag{2.23}$$

- **M-step**
  Si aggiornano i parametri con i valori ottimali dati gli attuali valori delle variabili latenti. Per θ_d:

$$\begin{cases} \dfrac{\partial \Lambda}{\partial \theta_{(d,z)}} = \displaystyle\sum_{d_i}^{N_d} \frac{< R_{(w_{d_i},z)} >}{\theta_{(d,z)}} - \lambda_d = 0 \\ \dfrac{\partial \Lambda}{\partial \lambda_d} = 1 - \displaystyle\sum_{z}^{T} \theta_{(d,z)} = 0 \end{cases} \tag{2.24}$$

  Risolvendo le due equazioni, si ottiene:

$$\theta_{(d,z)} = \frac{\sum_{d_i} < R_{(w_{d_i},z)} >}{N_d} \tag{2.25}$$

  Analogamente, per ϕ_z:

$$\begin{cases} \dfrac{\partial \Lambda}{\partial \phi_{(z,w)}} = \displaystyle\sum_{d}^{D} \sum_{d_i}^{N_d} \frac{< R_{(w_{d_i},z)} > \mathbb{I}(w_{d_i} = w)}{\theta_{(z,w)}} - \sigma_z = 0 \\ \dfrac{\partial \Lambda}{\partial \sigma_z} = 1 - \displaystyle\sum_{w}^{V} \phi_{(z,w)} = 0 \end{cases} \tag{2.26}$$

Risolvendo le due equazioni, si ottiene:

$$\phi_{(z,w)} = \frac{\sum_d^D \sum_{d_i}^{N_d} < R_{(w_{d_i},z)} > \mathbb{I}(w_{d_i} = w)}{\sum_{w'}^V \sum_d^D \sum_{d_i}^{N_d} < R_{(w_{d_i},z)} > \mathbb{I}(w_{d_i} = w')} \tag{2.27}$$

dove $\mathbb{I}(\cdot)$ è la funzione indicatrice.

2.3.1 Chiarimenti sull'algoritmo EM

Nella discussione precedente, c'è un dettaglio sottile che richiede ulteriore chiarimento. Abbiamo introdotto $R_{(w_{d_i},k)}$ come variabili indicatrici per specificare quale argomento (topic) viene scelto per la posizione di token d_i. Sebbene risulti che

$$\sum_k R_{(w_{d_i},k)} = 1,$$

questo vettore ha in realtà un solo elemento uguale a 1. Tuttavia, quando calcoliamo l'**E-step** dell'algoritmo di inferenza, determiniamo $< R_{(w_{d_i},k)} >$, ovvero la distribuzione a posteriori delle variabili nascoste, dato il dataset e i valori correnti dei parametri. In questo caso, $< R_{(w_{d_i},k)} >$ è una distribuzione che contiene probabilità in ciascun elemento del vettore, pur continuando a soddisfare

$$\sum_k < R_{(w_{d_i},k)} >= 1.$$

Che cosa causa esattamente questa differenza?

Riformuliamo la log-verosimiglianza di una singola posizione di token dopo aver introdotto le variabili indicatrici:

$$\log \sum_k^T R_{(w_{d_i},k)} \left(\phi_{(k,w_{d_i})} \, \theta_{(d,k)} \right) \tag{2.28}$$

Introduciamo una distribuzione ausiliaria $q(R_{(w_{d_i},k)}) = q(R_{(w_{d_i},k)} = 1)$, che soddisfa

$$\sum_k q(R_{(w_{d_i},k)}) = 1.$$

Sostituendo questa distribuzione ausiliaria nella log-verosimiglianza precedente, otteniamo:

$$\log \sum_k^T \frac{R_{(w_{d_i},k)}\, \phi_{(k,w_{d_i})}\, \theta_{(d,k)}}{q(R_{(w_{d_i},k)})}\, q(R_{(w_{d_i},k)}) = \log\, \mathbb{E}_q \left[\frac{R_{(w_{d_i},k)}\, \phi_{(k,w_{d_i})}\, \theta_{(d,k)}}{q(R_{(w_{d_i},k)})} \right] \tag{2.29}$$

Applicando la disuguaglianza di Jensen, possiamo spostare il logaritmo all'interno dell'aspettativa, ottenendo un limite inferiore alla log-verosimiglianza originale:

$$\log\, \mathbb{E}_q \left[\frac{R_{(w_{d_i},k)}\, \phi_{(k,w_{d_i})}\, \theta_{(d,k)}}{q(R_{(w_{d_i},k)})} \right] \geq \mathbb{E}_q \left[\log \frac{R_{(w_{d_i},k)}\, \phi_{(k,w_{d_i})}\, \theta_{(d,k)}}{q(R_{(w_{d_i},k)})} \right] \tag{2.30}$$

Il nostro obiettivo è quindi chiaro: poiché è difficile ottimizzare direttamente il termine a sinistra, dobbiamo massimizzare il limite inferiore (il termine a destra) il più possibile:

$$\begin{aligned} &\sum_k q\Big(R_{(w_{d_i},k)}\Big) \log \Big[R_{(w_{d_i},k)} \, \Big(\phi_{(k,w_{d_i})}\, \theta_{(d,k)} \Big) \Big] - \sum_k q\Big(R_{(w_{d_i},k)}\Big) \log q\Big(R_{(w_{d_i},k)}\Big) \\ &\quad + \lambda \left(1 - \sum_k q\Big(R_{(w_{d_i},k)}\Big) \right) \end{aligned} \tag{2.31}$$

Derivando rispetto a $q(R_{(w_{d_i},k)})$ e ponendo uguale a zero:

$$\log \Big[R_{(w_{d_i},k)}\, \phi_{(k,w_{d_i})}\, \theta_{(d,k)} \Big] - \log q(R_{(w_{d_i},k)}) - 1 - \lambda = 0 \tag{2.32}$$

Risolvendo, otteniamo:

$$q(R_{(w_{d_i},k)}) = \frac{\phi_{(k,w_{d_i})}\, \theta_{(d,k)}}{\sum_k^T \phi_{(k,w_{d_i})}\, \theta_{(d,k)}} \tag{2.33}$$

Questa è esattamente l'**E-step** ottenuta nella sezione precedente. Si osserva quindi che $q(R_{(w_{d_i},k)})$ coincide con $< R_{(w_{d_i},k)} >$, e si comprende che l'algoritmo EM, in questo contesto, può essere visto come un processo di massimizzazione di un limite inferiore.

Capitolo 3

Confronti sperimentali e applicazioni della PLSA

Nei capitoli precedenti sono state presentate le basi teoriche e matematiche necessarie alla comprensione del modello di *Probabilistic Latent Semantic Analysis* (PLSA), nonché la sua formulazione e le principali proprietà. In questo capitolo l'attenzione si sposta invece sugli aspetti **sperimentali e applicativi** del modello. Verranno riportati e discussi alcuni risultati presenti in letteratura [12, 13, 14], che consentono di valutare le prestazioni della PLSA rispetto alla *Latent Semantic Analysis* e di analizzare i vantaggi offerti dal punto di vista sia geometrico sia applicativo.
Oltre al confronto con LSA, saranno illustrate alcune applicazioni concrete della PLSA, come l'impiego in ambito di *Computer Vision* e l'implementazione su architetture GPU per aumentarne l'efficienza computazionale.

3.1 Confronto tra PLSA e LSA

Sebbene la *Latent Semantic Analysis* (LSA) e la *Probabilistic Latent Semantic Analysis* (PLSA) rappresentino documenti e termini in uno spazio semantico latente a dimensione ridotta, esse si basano su fondamenti teorici e metodologici molto diversi.

LSA, di natura algebrica, utilizza SVD per ottenere una rappresentazione compatta della matrice termine−documento, ottimizzando la ricostruzione mediante la

norma di Frobenius, che implica un'ipotesi di rumore gaussiano additivo sui dati. Questa ottimizzazione corrisponde a una proiezione ortogonale nello spazio latente.

Al contrario, la PLSA adotta un approccio probabilistico basato su un modello generativo esplicito e sul principio di massima verosimiglianza. L'obiettivo è massimizzare la capacità predittiva del modello, che equivale a minimizzare la cross-entropy, ovvero della **divergenza di Kullback–Leibler**, tra la distribuzione empirica dei dati e quella modellata. Questo consente un'interpretazione statistica chiara dei fattori latenti e una gestione più efficace di fenomeni come la sinonimia e la polisemia.

3.1.1 Confronto geometrico

Per comprendere meglio queste differenze, riscriviamo l'Aspect Model parametrizzato in (2.4) usando una notazione matriciale. Definiamo:

$$\hat{U} = \big(P(d_i|z_k)\big)_{i,k} \quad \hat{V} = \big(P(w_j|z_k)\big)_{j,k} \quad \hat{\Sigma} = \mathrm{diag}\big(P(z_k)\big)_k \tag{3.1}$$

dove $\hat{U}$ rappresenta la matrice delle distribuzioni condizionate dei documenti dati i fattori latenti, $\hat{V}$ quella delle parole condizionate ai fattori, e $\hat{\Sigma}$ la matrice diagonale delle proporzioni di mescolamento. Il modello di probabilità congiunta può quindi essere espresso come prodotto matriciale:

$$P = \hat{U}\hat{\Sigma}\hat{V}^\top \tag{3.2}$$

Confrontando questa decomposizione con la SVD utilizzata in LSA, emergono alcune analogie e differenze fondamentali:

- La somma dei prodotti esterni tra le righe di $\hat{U}$ e $\hat{V}$ riflette l'ipotesi di indipendenza condizionale nel modello probabilistico di PLSA.
- I K fattori corrispondono alle componenti di mescolamento del modello.
- Le proporzioni di mescolamento in PLSA assumono il ruolo analogo ai valori singolari nella SVD di LSA.

Dal punto di vista modellistico, ciò offre vantaggi significativi: la *mixture approximation* della tabella di co-occorrenza è una distribuzione di probabilità ben

definita e i fattori latenti hanno un'interpretazione probabilistica chiara. Al contrario, LSA non garantisce una distribuzione normalizzata e la matrice approssimata può contenere valori negativi, limitando l'interpretabilità. Inoltre, le direzioni nello spazio latente di LSA non hanno un significato immediato, mentre in PLSA corrispondono a distribuzioni multinomiali di parole, interpretabili come contesti tematici.

Un ulteriore vantaggio dell'approccio probabilistico è la possibilità di utilizzare la consolidata teoria statistica per la selezione del modello e il controllo della complessità, ad esempio per individuare il numero ottimale di dimensioni latenti. In LSA, invece, tale scelta si basa spesso su criteri empirici o euristici.

Per quanto riguarda la complessità computazionale, la SVD può essere calcolata esattamente (escludendo problemi di stabilità numerica), mentre l'algoritmo EM usato in PLSA è iterativo e garantisce solo il raggiungimento di un massimo locale della funzione di verosimiglianza. Tuttavia, negli esperimenti seguenti il tempo di calcolo di EM è risultato comparabile a quello della SVD sulla matrice di co-occorrenza. Inoltre, esistono ampi margini di miglioramento nelle prestazioni di EM tramite schemi di aggiornamento online, ancora poco esplorati.

3.1.2 Confronto applicativo: valutazione della *perplexity*

Per confrontare le prestazioni predittive di PLSA e LSA, è necessario specificare come estrarre le probabilità da una decomposizione LSA. Questo problema non è banale, poiché la presenza di valori negativi impedisce una semplice rinormalizzazione della matrice approssimante $\tilde{A}$. Pertanto, è stato adottato il metodo proposto in [14], che consente di ottenere probabilità più accurate rispetto a quelle derivabili da altre normalizzazioni più semplici.

Sono stati utilizzati due dataset per valutare le prestazioni in termini di *perplexity*:

- una collezione di test standard per il recupero dell'informazione, *MED*, contenente 1033 documenti;
- un dataset con coppie nome-aggettivo generato da una versione annotata del corpus *LOB*.

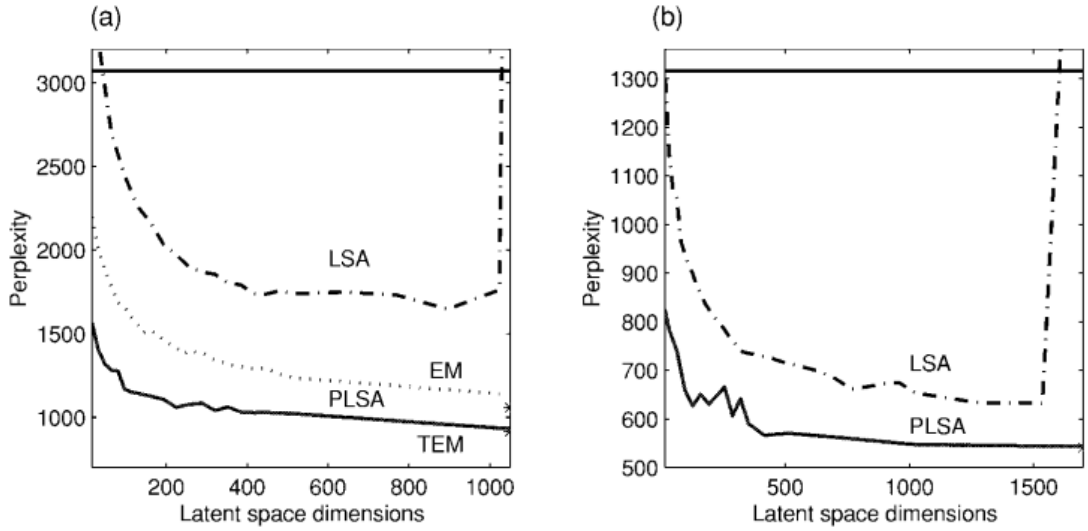


Figura 3.1: (a) dati MED (rango 1033); (b) dati LOB (rango 1674) [13]

In Figura 3.1 sono riportati i dati di *perplexity* per LSA e PLSA sui dataset MED e LOB in funzione del numero di dimensioni dello spazio semantico latente (probabilistico). Per le coppie nome-aggettivo, la *perplexity* riportata corrisponde alla predizione dei nomi condizionata all'aggettivo corrispondente.

Un aspetto interessante da osservare è il comportamento della curva relativa alla LSA per valori molto elevati di dimensionalità latente: come si nota dal tratteggio a destra, la *perplexity* cresce bruscamente invece di rimanere stabile o diminuire. Questo effetto è legato al fatto che, quando la dimensionalità si avvicina a quella originale dello spazio termine-documento, la LSA perde i benefici della riduzione dimensionale e tende a soffrire di fenomeni di overfitting, con un peggioramento della capacità di generalizzazione. La PLSA, al contrario, mostra un comportamento più regolare e robusto rispetto all'aumento della dimensionalità.

Il PLSA supera di gran lunga il modello statistico derivato da LSA standard: sulla collezione MED, PLSA riduce la *perplexity* rispetto al modello marginale di indipendenza di oltre un fattore tre ($3073/936 \approx 3.3$), mentre LSA ottiene una riduzione inferiore a un fattore due ($3073/1647 \approx 1.9$). Nei dati LOB, che sono meno sparsi, la differenza tra LSA e PLSA è meno marcata, ma comunque significativa: PLSA ottiene una riduzione di $1316/547 \approx 2.41$, mentre LSA si ferma a $1316/632 \approx 2.08$.

Per dimostrare i vantaggi del TEM, sono stati anche addestrati Aspect Model

sulla collezione MED utilizzando EM standard con *early stopping*.

Per "addestramento" si intende il processo con cui si insegna a un algoritmo di machine learning a rilevare gli schemi e a prevedere i risultati attraverso l'esposizione a dati etichettati.

Come si può osservare dalle curve in Figura 3.1(a), la differenza tra l'addestramento EM e TEM è significativa: sebbene entrambe le strategie (*annealing* e *early stopping*) siano efficaci nel controllare la complessità del modello, l'EM standard sfrutta in modo inefficiente i gradi di libertà disponibili.

Per approfondire l'impatto della scelta del parametro β e per evidenziare ulteriormente i vantaggi del TEM, sono stati effettuati una serie di esperimenti sul corpus TDT1 (Topic Detection and Tracking), utilizzando un campionamento casuale per analizzare l'effetto della dimensione della collezione di documenti. Poiché l'obiettivo era valutare il controllo dell'overfitting e non la presenza di massimi locali, tutti i modelli sono stati addestrati mantenendo la *temperature* costante.

La Figura 3.2 mostra le curve di *perplexity* per diversi valori della *inverse temperature* β in un modello con $K = 128$, in funzione del numero di iterazioni di TEM per vari livelli di sottocampionamento (*subsampling*). In tutti i casi, l'*early stopping* è stato applicato quando la *perplexity* sui dati di validazione aumentava.

Per gli esperimenti con sottocampionamento 32x ($N = 496$ documenti), le esecuzioni sono ripetute 5 volte per valutare la variabilità delle soluzioni rispetto a diverse condizioni iniziali casuali. Osserviamo che:

- Sebbene l'*early stopping* prevenga l'overfitting, il controllo tramite *temperature* produce un miglioramento significativo e costante per tutte le dimensioni campionarie.

- I vantaggi del TEM sono notevoli anche per l'intero dataset TDT1 con 7 milioni di token: l'overfitting rimane quindi un problema anche nei dataset di grandi dimensioni.

- La complessità computazionale del TEM è leggermente superiore rispetto all'EM standard ($\beta = 1$), richiedendo tipicamente il doppio delle iterazioni per convergere.

- La variabilità nella qualità delle soluzioni dovuta a diverse condizioni iniziali è ridotta rispetto alla differenza di *perplexity* tra EM e TEM.

- Sebbene β sia un parametro critico, esiste un ampio intervallo di valori per cui si ottengono prestazioni simili; il TEM è quindi piuttosto robusto rispetto alla scelta del valore ottimale di β.

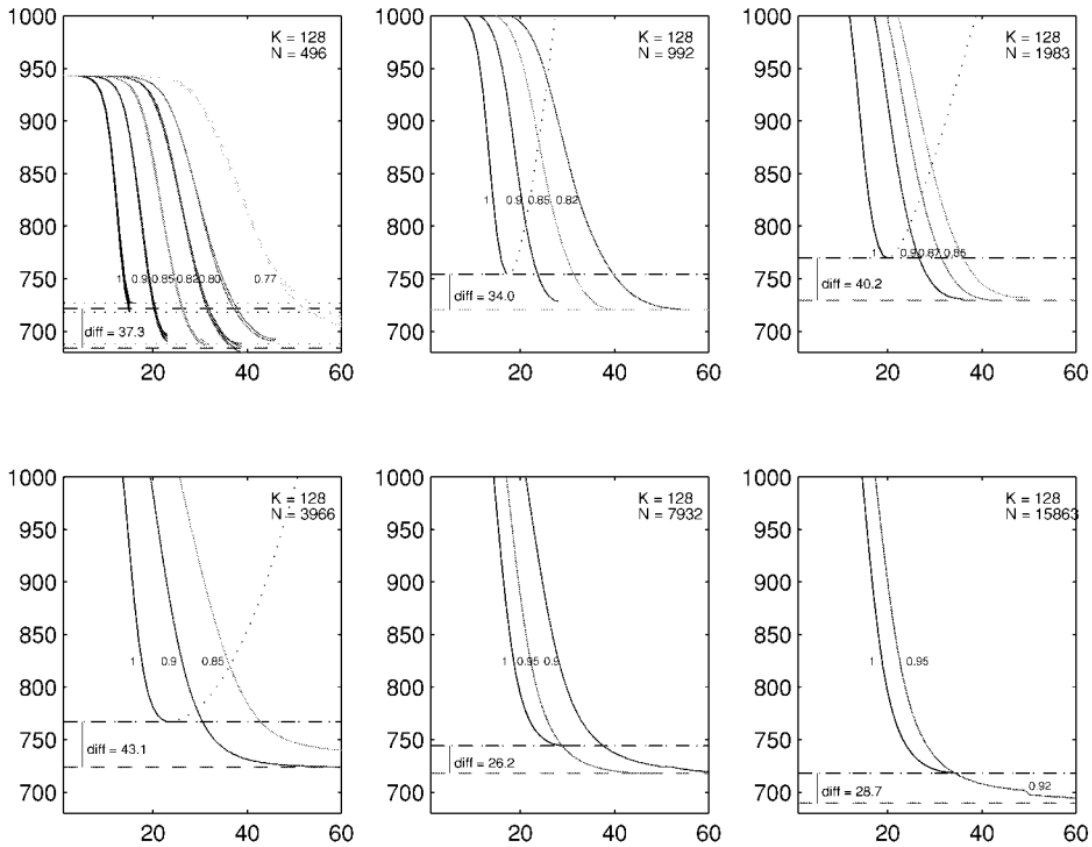


Figura 3.2: Esperimenti di annealing sul dataset TDT1 con un modello a $K = 128$ aspetti e diversi fattori d di sottocampionamento (da sinistra in alto a destra in basso: 32x, 16x, 8x, 4x, 2x, 1x). N indica il numero di documenti nel set di addestramento, mentre i numeri piccoli rappresentano la *inverse temperature* β utilizzata per le curve di apprendimento. La linea tratteggiata mostra le prestazioni dell'algoritmo EM oltre il punto di arresto normale [13].

3.1.3 Information retieval con PLSA e LSA

I risultati riportati in [13] mostrano un confronto tra la LSA e la PLSA nell'ambito dell'*Information Retrieval*, con particolare attenzione alle prestazioni delle due tecniche in relazione al recupero basato su query. Uno dei principali problemi

dell'*Information Retrieval* è l'indicizzazione automatica, la cui applicazione principale si trova nel recupero basato su query. La famiglia di tecniche più diffusa è quella basata sul *Vector-Space Model* (VSM) per i documenti.

Una variante del VSM è caratterizzata da tre elementi: una funzione di trasformazione (detta *local term weight*), uno schema di pesatura dei termini (*global term weight*) e una misura di similarità. Nel presente lavoro, al fine di confrontare le prestazioni relative di *direct term matching*, LSA e PLSA, è stata scelta una rappresentazione semplice, basata sulle frequenze non trasformate dei termini $n(d_i, w_j)$ e sulla misura di similarità coseno standard. La stessa rappresentazione utilizzata per i documenti viene applicata anche alle query q, così che la funzione di similarità per il metodo di base può essere scritta come:

$$s(d_i, q) = \frac{\sum_{j=1}^{M} n(d_i, w_j) n(q, w_j)}{\sqrt{\sum_{j=1}^{M} n(d_i, w_j)^2} \sqrt{\sum_{j=1}^{M} n(q, w_j)^2}} \tag{3.3}$$

Nel *Latent Semantic Indexing* (LSI), la rappresentazione originale dei documenti dello spazio vettoriale viene sostituita con una rappresentazione nello spazio latente a bassa dimensionalità e la similarità calcolata in questo nuovo spazio. Query o documenti non presenti nella collezione originale possono essere inseriti (*folded in*) mediante una semplice moltiplicazione di matrici. Nei seguenti esperimenti, sono state considerate combinazioni lineari tra lo score di similarità originale (peso λ) e quello derivato dalla rappresentazione dello spazio latente (peso $1 - \lambda$).

Le stesse idee sono state applicate nel *Probabilistic Latent Semantic Indexing* (PLSI) in combinazione con il modello PLSA. Più precisamente, sono state utilizzate le rappresentazioni a bassa dimensionalità nello spazio dei fattori $P(z_k|d_i)$ e $P(z_k|q)$ per calcolare la similarità. Per fare ciò, le query devono essere inserite, operazione che in PLSA viene eseguita fissando i parametri $P(w_j|z_k)$ e calcolando i pesi $P(z_k|q)$ tramite TEM.

Un vantaggio nell'utilizzo di modelli statistici rispetto alle tecniche basate sulla SVD è la possibilità di combinare in modo sistematico più modelli. Sebbene idealmente questa combinazione dovrebbe essere effettuata seguendo uno schema bayesiano, negli esperimenti presentati è stato adottato un approccio più semplice che, nonostante la sua semplicità, ha dimostrato ottime prestazioni e buona robustezza: gli score coseno di tutti i modelli sono stati combinati con pesi uniformi.

Il metodo risultante viene denominato PLSI*. Da un punto di vista empirico, si è osservato che le prestazioni risultano molto stabili rispetto a diverse scelte di pesi, anche non uniformi, e rispetto al valore di λ impiegato nella combinazione con lo score coseno originale, grazie agli effetti benefici di riduzione del rumore derivanti dall'aggregazione dei modelli.

Si noti che le rappresentazioni LSA per diversi valori di K formano una sequenza annidata, mentre questo non è vero per modelli statistici, che si prevede catturino una maggiore varietà di decomposizioni ragionevoli.

Sono state utilizzate le seguenti collezioni di documenti di dimensione media, con valutazioni di rilevanza:

1. **MED**: 1033 abstract di documenti della *National Library of Medicine*
2. **CRAN**: 1400 abstract di documenti su aeronautica del *Cranfield Institute of Technology*
3. **CACM**: 3204 abstract degli articoli della rivista *CACM*
4. **CISI**: 1460 abstract di articoli di scienze bibliotecarie dell'*Institute for Scientific Information*

Per tutte le collezioni, le query sono annotate con *ground truth*, ovvero un insieme di documenti rilevanti determinati da esperti umani. I risultati sintetici, in termini di *average precision* (calcolata sui 9 livelli di richiamo dal 10% al 90%), sono riportati nella Tabella 3.1, mentre le curve *precision-recall* corrispondenti si trovano in Figura 3.3

	MED		CRAN		CACM		CISI	
	prec.	impr.	prec.	impr.	prec.	impr.	prec.	impr.
cos + tf	44.3	–	29.9	–	17.9	–	12.7	–
LSI	51.7	+16.7	*28.7	−4.0	*16.0	−11.6	12.8	+0.8
PLSI	63.9	+44.2	35.1	+17.4	22.9	+27.9	18.8	+48.0
PLSI*	66.3	+49.7	37.5	+25.4	26.8	+49.7	20.1	+58.3

Tabella 3.1: Risultati di *average precision* e *relative improvement* rispetto al metodo baseline *cos + tf* per le quattro collezioni di test standard. Un asterisco accanto a LSI indica che non è stato possibile ottenere un miglioramento rispetto al baseline [13]

Altri dettagli sperimentali: i modelli PLSA con $K = 32, 48, 64, 80, 128$ sono stati addestrati con TEM su ciascun insieme di dati, utilizzando il 10% dei dati come

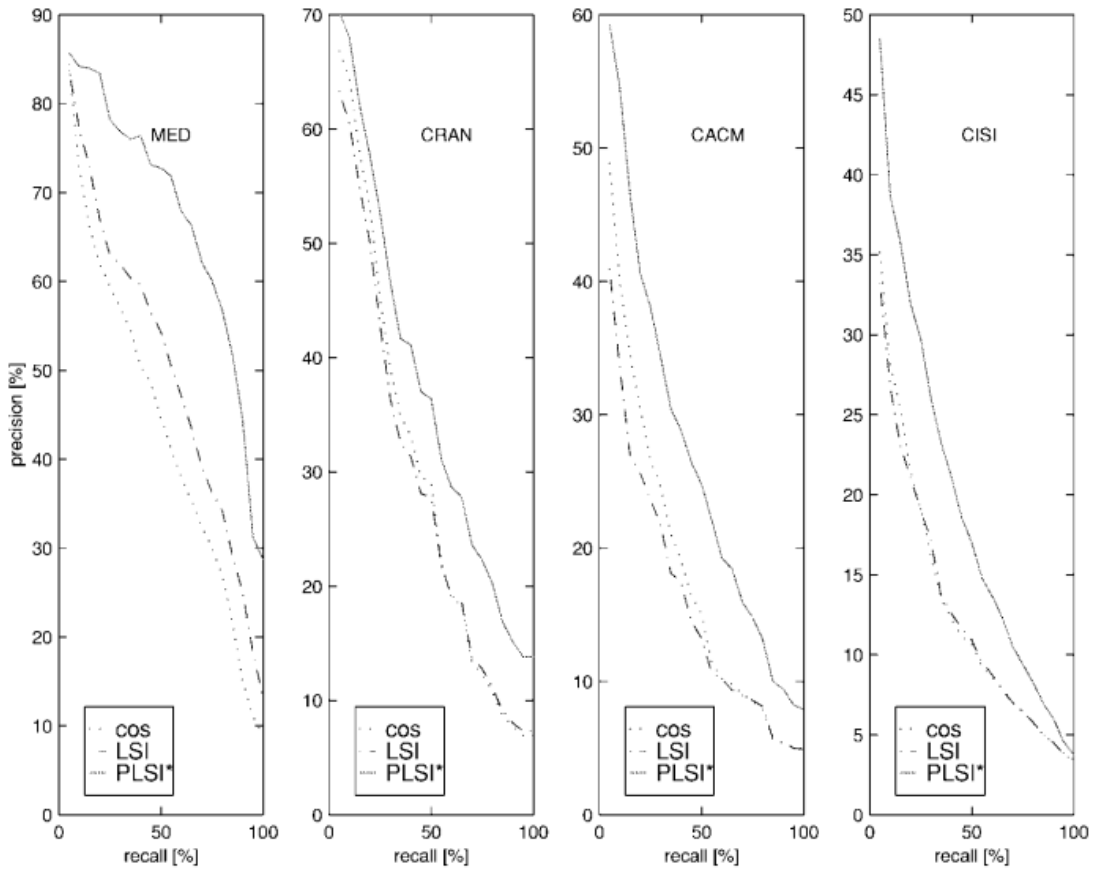


Figura 3.3: Curve *precision-recall* per le 4 collezioni di test. Sono riportate le curve relative al *direct term matching*, LSI e PLSI* [13].

hold-out. Per PLSI è stato riportato il miglior risultato ottenuto da uno di questi modelli; per LSI è stato riportato il miglior risultato ottenuto per la dimensione ottimale (esplorando da 32 a 512 dimensioni con passo 8). Il peso λ per la combinazione con lo score coseno di base è stato ottimizzato manualmente: MED, CRAN $\lambda = \frac{1}{2}$; CACM, CISI $\lambda = \frac{2}{3}$. In generale, per i modelli combinati sono stati utilizzati pesi leggermente più piccoli, sebbene i risultati siano molto robusti rispetto alla scelta precisa di λ.

Gli esperimenti confermano costantemente i vantaggi di PLSI su LSI. Miglioramenti sostanziali delle prestazioni sono stati ottenuti su tutti e quattro gli insiemi di dati. Si noti che il guadagno relativo in *precision* rispetto al metodo di base è solitamente intorno al 100% nella fascia intermedia di *recall*, che è quella più interessante. In particolare, PLSI funziona bene anche quando LSI fallisce completamente.

Questi esperimenti dimostrano chiaramente che i vantaggi di PLSA rispetto a LSA non si limitano ai casi in cui le metriche di valutazione siano direttamente correlate alla *perplexity*. Di conseguenza, funzioni obiettivo di natura statistica, come la *perplexity* (log-verosimiglianza), possono essere considerate un criterio di riferimento generale per l'analisi nei contesti di *text learning* e *Information Retrieval*.

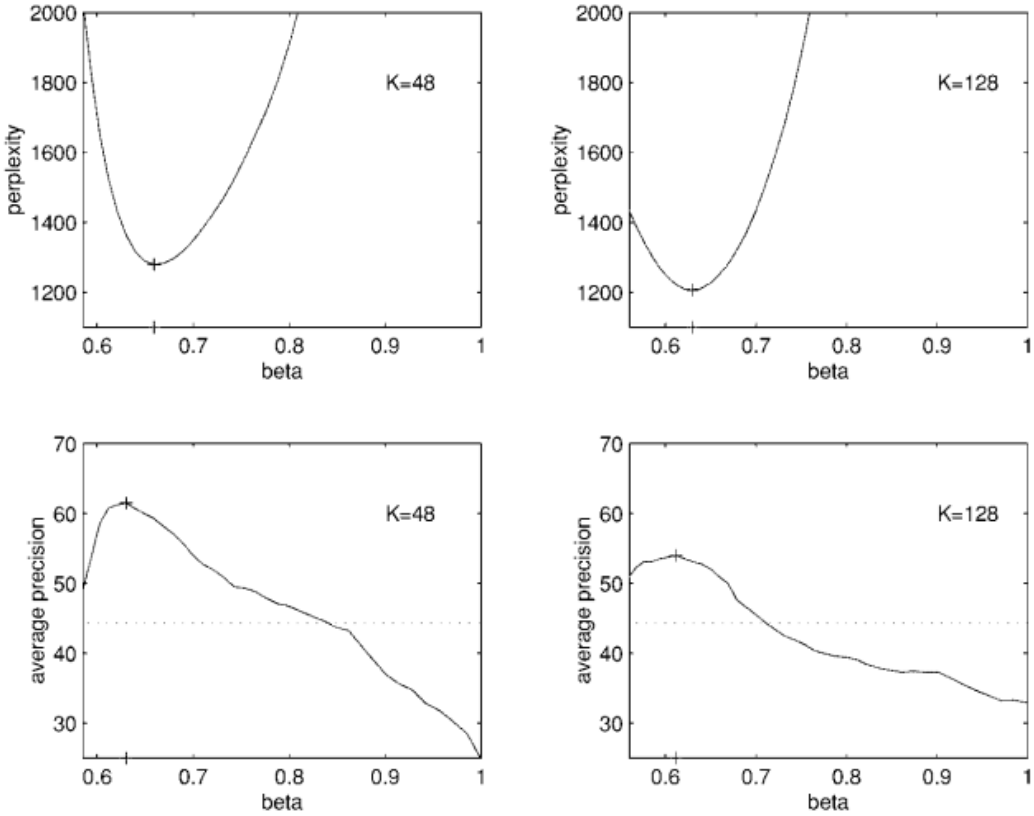


Figura 3.4: *Perplexity* e *average precision* in funzione della *inverse temperature* β per un Aspect Model con $K = 48$ (a sinistra) e $K = 128$ (a destra) [13]

Per enfatizzare questo aspetto, sono stati condotti una serie di esperimenti in cui sia la *perplexity* sia la *average precision* sono state monitorate simultaneamente come funzione di β. Le curve risultanti per la collezione MED sono riportate in Figura 3.4; i risultati sulle altre collezioni sono molto simili. Sebbene le due curve non raggiungano i rispettivi estremi esattamente per lo stesso valore di β, la correlazione è piuttosto evidente: le migliori prestazioni in *retrieval* si ottengono per valori di β leggermente inferiori a quello determinato sui dati di *hold-out*.

Varianti del *Probabilistic Latent Semantic Indexing*

Per approfondire ulteriormente le potenzialità del modello, sono state proposte alcune varianti del *Probabilistic Latent Semantic Indexing* (PLSI) specificamente pensate per il compito di indicizzazione. L'idea alla base di queste estensioni è quella di sfruttare in maniera più flessibile le rappresentazioni probabilistiche fornite dal modello, sia nel dominio originale delle parole sia nello spazio latente dei fattori. Tra le diverse possibilità discusse in letteratura, in particolare in [12], sono stati analizzati due approcci principali:

1. un **context-dependent unigram model**, volto a rendere più fluide le distribuzioni empiriche delle parole nei documenti (PLSI-U);

2. un modello dello spazio latente che fornisce una rappresentazione a bassa dimensionalità di documenti e query (PLSI-Q).

PLSI-U: Per ciascun documento d nella collezione, il PLSA fornisce una distribuzione multinomiale $P(w_j|d)$ sul vocabolario, come definito in (2.2). Questa distribuzione sarà in generale una versione *smooth* della distribuzione empirica

$$\hat{P}(w_j|d) = \frac{n(d,w)}{n(d)}$$

Si propone di utilizzare $P(w_j|d)$ (inteso come un vettore documento) per calcolare uno score di corrispondenza tra un documento ed una query. Si noti che $P(w_j|d)$ è una rappresentazione nello spazio originale delle parole, ottenuta per proiezione inversa dallo spazio latente probabilistico.

Il vettore $P(\cdot|d)$ può essere eventualmente pesato con le *inverse document frequencies* e confrontato con la query, anch'essa pesata, tramite il coseno al quadrato.

Sono stati considerati due modi di combinare PLSI-U con lo standard VSM:

1. combinando linearmente le similarità coseno, come discusso in precedenza per l'LSI,

2. combinando additivamente le multinomiali, come nei metodi di interpolazione per il *language modeling*, ovvero utilizzando la rappresentazione:

$$\tilde{P}(w_j|d) = \lambda\, \hat{P}(w_j|d) + (1-\lambda)P(w_j|d)$$

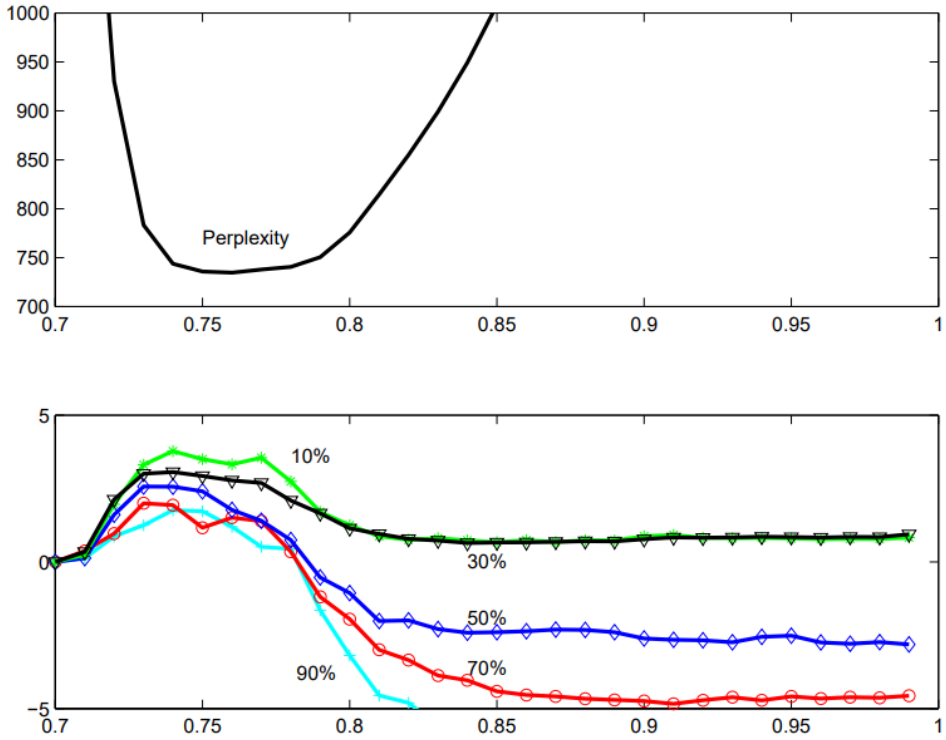


Figura 3.5: Prestazioni del modello con $K = 256$ sulla collezione Cranfield in termini di *perplexity* (grafico superiore) e *precision* (grafico inferiore) a diversi valori di β. Il modello è stato *annealed* ($0.7 \rightarrow \beta \rightarrow 1.0$) e addestrato fino alla convergenza; non è stato applicato alcun *early stopping* [12]

Entrambi i metodi hanno mostrato prestazioni empiricamente quasi identiche; pertanto, sono stati riportati solo i risultati della prima variante, poiché questo schema è stato adottato anche nel caso del LSI.

PLSI-Q: In questo schema è stata utilizzata la rappresentazione a bassa dimensione $P(z|d)$ e $P(z|q)$ per valutare le similarità. Di conseguenza, le query devono essere *folded in*, cosa che si realizza fissando i parametri $P(w|z)$ e calcolando i pesi $P(z|q)$ tramite TEM.
Come considerare in modo ottimale i pesi (globali) dei termini in PLSI-Q è un problema risolto parzialmente. È stato utilizzato un approccio ad hoc per ripesare i diversi componenti del modello tramite le quantità

$$\sum_w P(w|z) \cdot \mathrm{idf}(w),$$

dove idf(w) (*inverse document frequency*) è una misura dell'importanza del termine w calcolata in base alla sua rarità nei documenti. Tuttavia, ciò potrebbe non sfruttare in modo ottimale la priorità dei pesi dei termini.

Uno dei vantaggi nell'usare modelli statistici rispetto alle tecniche SVD è che consente di combinare sistematicamente modelli diversi. Sebbene ciò dovrebbe essere fatto in modo ottimale secondo uno schema bayesiano di combinazione di modelli, negli esperimenti è stato utilizzato un approccio molto più semplice che comunque ha mostrato ottime prestazioni e robustezza.

Nel PLSI-U sono state combinate le stime di probabilità $P(w|d)$ per modelli con diverso numero di componenti K in modo additivo con pesi uniformi. Nello schema PLSI-Q, sono stati combinati i punteggi coseno di tutti i modelli con un peso uniforme. I metodi risultanti sono denominati rispettivamente PLSI-U* e PLSI-Q*.

Empiricamente è stato riscontrato che le prestazioni sono molto robuste rispetto a pesi diversi (non uniformi) e anche rispetto al peso λ usato in combinazione con il punteggio coseno originale. Ciò è dovuto ai benefici di riduzione del rumore derivanti dall'*averaging* (media) dei modelli.

Da notare che le rappresentazioni LSA per diversi valori di K formano una sequenza annidata, cosa che non vale per i modelli statistici, i quali sono invece in grado di catturare una più ampia varietà di decomposizioni ragionevoli.

3.2 Applicazione della PLSA alla Computer Vision

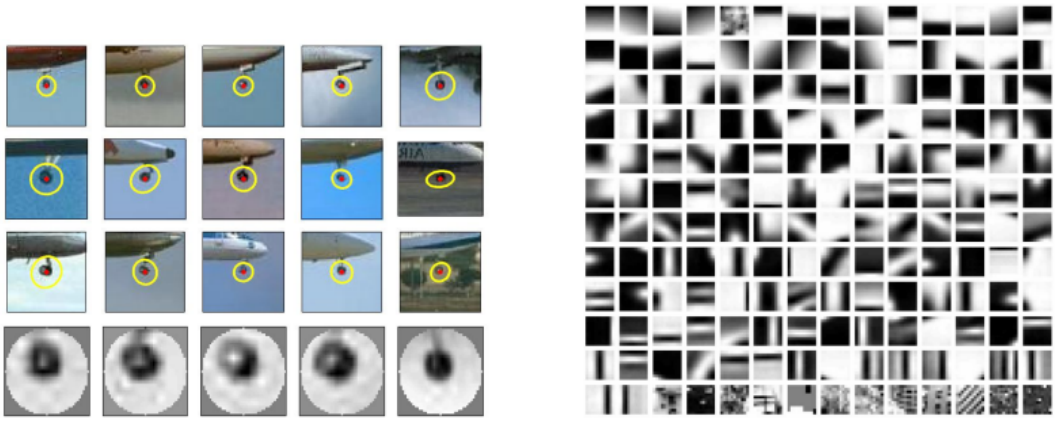
Una delle estensioni del modello PLSA per applicazioni di Computer Vision è stata proposta in [16, 17]. Questi lavori propongono l'utilizzo del modello PLSA su insiemi di immagini per estrarre categorie di oggetti in modo non supervisionato, per poi classificare nuove immagini grazie agli oggetti appresi e segmentare immagini raggruppando insieme caratteristiche locali appartenenti a un determinato oggetto.

È stato utilizzato il framework standard di PLSA; tuttavia, invece che su documenti, l'algoritmo operava su immagini, le parole sono state sostituite da patch/parole visive, e il topic era rappresentato da una categoria di oggetto. La differenza più marcata rispetto ai casi precedenti è che le "parole" non sono chiaramente specificate per un insieme di immagini. Queste possono essere ottenute in tre fasi:

1. **Rilevamento delle feature**: vengono estratte regioni ellittiche usando un rilevatore di punti di interesse affine-invariante;

2. **Rappresentazione delle feature**: le patch precedentemente rilevate sono scalate a cerchi (vedi Figura 3.6a, ultima riga) e viene calcolato il descrittore *SIFT* (Scale-Invariant Feature Transform);

3. **Generazione del codebook**: dato l'enorme numero di parole visive risultanti (circa centinaia di migliaia), il loro numero viene ridotto tramite vettorializzazione (vector quantization) a circa 2000 – le parole vengono raggruppate usando l'algoritmo *k-means* e ogni cluster è rappresentato dal suo centroide (vedi Figura 3.6b). Così si determina un dizionario di "parole" e ogni immagine può essere rappresentata come un "sacco" di queste patch (modello *Bag-of-Words*); di conseguenza l'intero dataset può essere rappresentato come una matrice di co-occorrenza.



(a) Esempi di una "parola" visuale (b) Esempio di dizionario

Figura 3.6: I passaggi eseguiti per ottenere l'insieme delle parole in codice [16]

Di seguito, descriviamo come il modello PLSA sia stato applicato in diversi scenari:

- **Categorizzazione degli oggetti**: il processo di training di PLSA produce le probabilità $P(z|d)$ e $P(w|z)$; usando $P(z|d_n)$ per ogni immagine d_n, le immagini sono classificate come contenenti l'oggetto k, dove $k = \arg\max_{z_k \in Z} P(z_k|d_n)$.

- **Classificazione di immagini non viste**: le nuove immagini sono classificate usando la tecnica *fold-in*. Prima si esegue la procedura standard di training (l'algoritmo EM) sul dataset. Quando si ha una nuova immagine query d_{new}, l'algoritmo di training viene rieseguito, ma questa volta $P(w|z)$ rimane fissato ai valori precedenti, mentre viene aggiornato solo $P(z|d_{new})$. In questo modo si ottengono i coefficienti di mixing $P(z|d_{new})$ per l'immagine non vista.

- **Segmentazione**: la segmentazione spaziale può essere ottenuta usando la distribuzione posteriore:

$$P(z|w,d) = \frac{P(w|z,d)P(z|d)}{P(w|d)} = \frac{P(w|z)P(z|d)}{\sum_{z \in Z} P(w|z)P(z|d)} \tag{3.4}$$

  Questa fornisce la probabilità che ogni parola in un'immagine sia generata da un certo topic

3.2.1 Auto-annotazione

In [18, 19] è stato affrontato il problema dell'auto-annotazione delle immagini con il modello PLSA. Nel loro primo lavoro, è stato utilizzato un sistema misto PLSA in cui ogni documento è rappresentato da una coppia immagine-annotazione e le parole sono una concatenazione di parole visive testuali (presenti nell'annotazione). Questo approccio si basa sul fatto che i topic latenti sono gli stessi sia per le parole visive che per il testo. Tuttavia, per dataset in cui la modalità visiva non corrisponde a quella testuale, questo metodo presenta delle limitazioni; ciò accade perché se si adottano due modelli PLSA, uno sulle immagini e l'altro sul testo, questi imparano topic differenti. Nel lavoro successivo, sono stati utilizzati due modelli PLSA collegati che condividono la distribuzione sui topic $P(z|d)$. Il processo di apprendimento avviene in due fasi: prima viene adattato un modello solo alle informazioni testuali e poi l'altro modello utilizza $P(z|d)$ ottenuto precedentemente per imparare la distribuzione sulle parole visive $P(\text{visual words}|z)$. In questo modo viene garantita la coerenza semantica.

3.3 PLSA su GPU: un approccio efficiente

Le GPU (*Graphics Processing Unit*) sono ampiamente utilizzate per migliorare le prestazioni in vari ambiti, dalla simulazione fisica all'elaborazione di segnali e immagini, fino alle applicazioni in database e data mining. L'uso di CUDA (*Compute Unified Device Architecture*) ha permesso di sviluppare tecniche per accelerare query, analisi testuali e ordinamenti di grandi volumi di dati. Tuttavia, l'impiego delle GPU per l'analisi di testi è ancora poco esplorato, soprattutto a causa delle sfide legate alla grande dimensione dei dati testuali e alla limitata memoria disponibile. In [20] è presentato uno dei lavori collegati in questa categoria: si propone di raggruppare grandi insiemi di dati usando CUDA. Molti altri studi sono riportati nella survey sulle GPU [21].

3.3.1 Accelerazione dell'algoritmo EM tramite moltiplicazioni matrice-vettore

Una delle caratteristiche delle GPU è la capacità di eseguire operazioni SIMD (*Single Instruction, Multiple Data*), come la moltiplicazione vettoriale, in modo estremamente efficiente grazie all'elevato parallelismo. Per questo motivo, nell'implementazione di PLSI è fondamentale sfruttare al meglio tali meccanismi per ottenere migliori prestazioni.

In particolare, in [22] è stato riorganizzato l'algoritmo EM in modo da utilizzare l'operazione di moltiplicazione vettoriale. Ciò è vantaggioso poiché consente di impiegare pacchetti di calcolo numerico già pronti, come CUBLAS, nell'implementazione.

La proposta si basa sulle seguenti osservazioni: analizzando l'algoritmo EM, si possono classificare i termini presenti nella formula in due gruppi:

- componenti che dipendono dalle iterazioni
- componenti che dipendono dalle classi latenti

Nel primo gruppo rientrano i componenti che restano invariati durante le iterazioni e che sono condivisi da tutte le classi latenti; nel secondo, invece, quelli che variano in base alla classe latente considerata.

Partendo da questa distinzione, l'algoritmo EM viene riorganizzato separando le operazioni e fondendo alcuni passaggi, così da incrementare l'efficienza sia in termini di tempo di calcolo sia di utilizzo della memoria. Le equazioni sono inoltre formulate direttamente in forma matriciale e vettoriale, in modo da eseguire i calcoli in maniera più rapida ed efficace.

Introduciamo la notazione matriciale per i termini delle formule precedenti:

- $P(z) = Z \in \mathbb{R}^{K \times K}$, matrice delle probabilità a priori sui topic;
- $P(d|z) = B \in \mathbb{R}^{K \times D}$, matrice delle distribuzioni dei documenti condizionate ai topic;
- $P(w|z) = C \in \mathbb{R}^{K \times W}$, matrice delle distribuzioni delle parole condizionate ai topic.

Si supponga inoltre che l'elemento (x, y) di una matrice M sia indicato come $M_{x,y}$. Allora, nell'equazione (2.4), $\Gamma(d_x, w_y)$ (corrispondente a $P(d_x, w_y)$ in forma matriciale) è calcolato come:

$$\Gamma(d_x, w_y) = \left((ZB)^\top C\right)_{x,y} \tag{3.5}$$

Il denominatore $\Gamma(d, w)$ è comune a tutte le classi latenti in ogni iterazione ed è rappresentato come una matrice $D \times W$.

Il numeratore, invece, varia in base alla classe $k \in \{1,2, \ldots, K\}$ ed è calcolato come:

$$\Psi_k(d_x, w_y) = \left((Z_k B)^\top C\right)_{x,y} \tag{3.6}$$

dove Z_k indica la probabilità della classe latente k, mentre B e C forniscono rispettivamente le distribuzioni sui documenti e sulle parole condizionate a z_k. In questo caso, $B_{k,:}$ e $C_{k,:}$ rappresentano i vettori di dimensione D e W corrispondenti alle distribuzioni di d e w sotto la classe z_k.

Infine, l'**E-step** si calcola a partire dal denominatore Γ e dal numeratore Ψ. Per ridurre il carico computazionale, ciascuna coppia (d_i, w_j) è stata moltiplicata direttamente per la propria occorrenza $n(d_i, w_j)$ già nel calcolo dell'aspettativa, poiché la probabilità dell'equazione (2.4) non viene mai utilizzata isolatamente, ma sempre pesata con $n(d_i, w_j)$.

$$
\begin{array}{cccccc}
\frac{\Psi(d_1,w_1)}{\Gamma(d_1,w_1)}n(d_1,w_1) & \frac{\Psi(d_1,w_2)}{\Gamma(d_1,w_2)}n(d_1,w_2) & \dots\ \dots\ \dots & \frac{\Psi(d_1,w_{W-1})}{\Gamma(d_1,w_{W-1})}n(d_1,w_{W-1}) & \frac{\Psi(d_1,w_W)}{\Gamma(d_1,w_W)}n(d_1,w_W) \\
\frac{\Psi(d_2,w_1)}{\Gamma(d_2,w_1)}n(d_2,w_1) & \frac{\Psi(d_2,w_2)}{\Gamma(d_2,w_2)}n(d_2,w_2) & \dots\ \dots\ \dots & \dots & \frac{\Psi(d_2,w_W)}{\Gamma(d_2,w_W)}n(d_2,w_W) \\
\dots & \dots & \dots\ \dots\ \dots & \dots & \dots \\
\frac{\Psi(d_{D-1},w_1)}{\Gamma(d_{D-1},w_1)}n(d_{D-1},w_1) & \frac{\Psi(d_{D-1},w_2)}{\Gamma(d_{D-1},w_2)}n(d_{D-1},w_2) & \dots\ \dots\ \dots & \frac{\Psi(d_{D-1},w_{W-1})}{\Gamma(d_{D-1},w_{W-1})}n(d_{D-1},w_{W-1}) & \dots \\
\frac{\Psi_{D1}(d_D,w_1)}{\Gamma(d_D,w_1)}n(d_D,w_1) & \frac{\Psi(d_D,w_2)}{\Gamma(d_D,w_2)}n(d_D,w_2) & \dots\ \dots\ \dots & \frac{\Psi(d_D,w_{W-1})}{\Gamma(d_D,w_{W-1})}n(d_D,w_{W-1}) & \frac{\Psi(d_D,w_W)}{\Gamma(d_D,w_W)}n(d_D,w_W)
\end{array}
$$

Figura 3.7: Step finale di **Expectation** moltiplicato per l'occorrenza della parola [22]

L'equazione (3.7) riporta lo step E per una classe o:

$$P(z_o|d_x, w_y) = \frac{(B^\top C)_{x,y}}{\left((ZB)^\top C\right)_{x,y}} n(d_x, w_y) \tag{3.7}$$

Per i dettagli dell'implementazione, si sfrutta il metodo di *parallel array reduction*. Inoltre, è stata proposta una versione ottimizzata per CUDA, la cui implementazione è disponibile nella libreria CUDPP.

Nello **step M** questa libreria viene utilizzata nel seguente modo:

1. riduzione dell'intera matrice per normalizzare le probabilità;

2. *row multi-scan* per calcolare le probabilità dei documenti;

3. *row multi-scan* per trasporre la matrice delle aspettative, così da calcolare le probabilità delle parole.

Dopo aver calcolato le aspettative, il passo successivo è lo step M. Questo può essere implementato in modo efficiente su GPU, poiché non richiede cicli:

- un thread si occupa di aggiornare la probabilità di un documento,

- un altro thread aggiorna la probabilità di una parola.

Queste operazioni sono eseguite in parallelo per ciascuna classe latente.

3.3.2 Sfruttare la sparsità dei dati

Nella pratica, la matrice illustrata in Figura 3.7 risulta piuttosto sparsa. La Tabella 3.2 riporta le statistiche provenienti dai dataset di addestramento utilizzati negli esperimenti precedenti.

Dataset	#Docs	#Words	#Occurrence	Occupancy
CRAN	1398	7867	63813	0.58%
MED	1033	10062	28013	0.32%
CISI	1460	9765	34947	0.29%

Tabella 3.2: Statistiche dei dataset utilizzati negli esperimenti [22]

Poiché la memoria totale disponibile su GPU è strettamente limitata, è fondamentale adottare tecniche di rappresentazione di matrici sparse per gestire dataset di grandi dimensioni. Di seguito verrà adottato il formato **Compressed Sparse Row** (CSR) per la sua semplicità ed efficienza in termini di memoria.

Il formato CSR memorizza gli indici di colonna e i valori non nulli in array separati come indici e dati, mentre le informazioni di riga vengono memorizzate in un array di puntatori denominato *ptr*. Ad esempio, in una matrice $D \times W$, *ptr* ha lunghezza $D + 1$, mentre *indices* e *data* hanno dimensione pari al numero di elementi non nulli nella matrice.

3.3.3 Ottimizzazione sull'architettura Fermi

Nella versione modificata dell'algoritmo EM è possibile separare le componenti dipendenti dalla classe da quelle indipendenti dalla classe. Analizzando più da vicino la parte dipendente dalla classe, si osserva che il calcolo delle aspettative e l'aggiornamento dei parametri relativi a una classe non interferiscono con quelli di altre classi. Questo apre la possibilità a un'ulteriore ottimizzazione.

L'architettura **NVIDIA Fermi**, offre la possibilità di eseguire *kernel* in parallelo. In questo modo è possibile eseguire processi in parallelo in funzione del numero di classi latenti. Per evitare che la memoria globale venga sovrascritta, il sistema è progettato affinché ogni *kernel* disponga di un proprio spazio di lavoro. Tuttavia, questa soluzione non è efficiente in termini di memoria e ne limita l'applicazione a dataset di dimensioni moderate.

3.3.4 Esperimenti

È stata condotta una serie di esperimenti per dimostrare l'efficacia del metodo. In particolare, si confronta l'implementazione basata su CPU con la versione GPU.

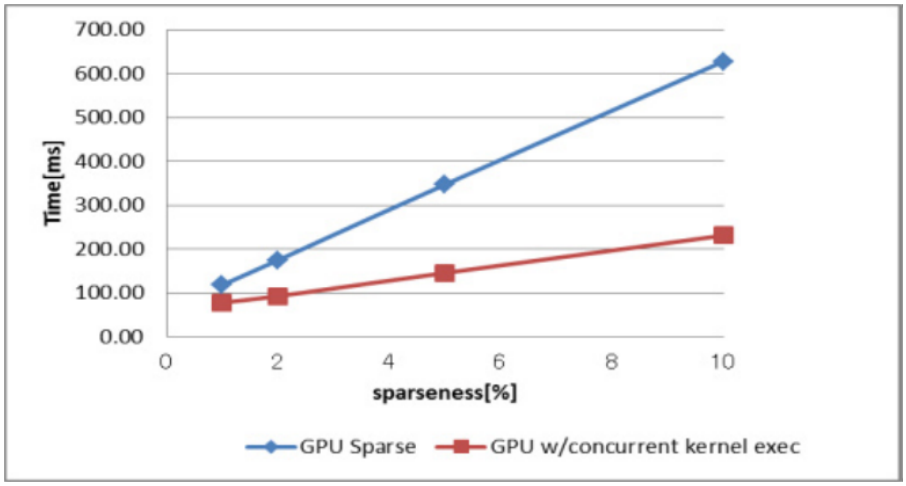
Dataset

Sono stati utilizzati sia dati sintetici sia dati reali, prendendo come riferimento i set di addestramento CRAN, MED e CISI. Al set di addestramento è stato applicato uno *stemming standardizzato* e l'eliminazione delle *stop-word* al set di addestramento per ridurre il rumore. Le specifiche dei dataset, come il numero di documenti e il numero di parole, sono riassunte nella Tabella 3.2. Sono stati generati inoltre dati sintetici per testare lo schema proposto in ambienti controllati; è stato fissato il numero dei documenti e di parole per documento, mentre è stata variata la percentuale di elementi non nulli nei documenti. Anche il numero di classi latenti è stato modificato per valutarne l'impatto.

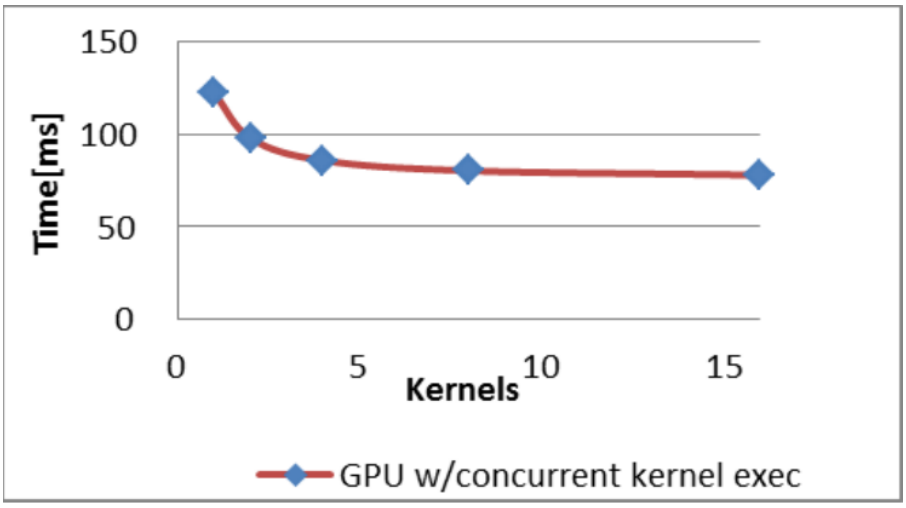
Risultati sperimentali

Usando dati sintetici, sono stati condotti due esperimenti distinti. Nel primo, è stato testato l'impatto della *sparseness* sul tempo di esecuzione variando il numero di elementi non nulli nella matrice di occorrenza. Il numero di classi latenti è stato fissato a 32, la matrice di occorrenza ha dimensione 1000×5000 e il numero di *kernel* concorrenti è stato impostato a 16. In Figura 3.8a si osserva che il numero di elementi non nulli influenza il tempo di esecuzione.

Nel secondo esperimento, è stato misurato il tempo di esecuzione variando il numero di *kernel* concorrenti. NVIDIA consente l'esecuzione contemporanea di massimo 16 *kernel*. È stata evidenziata la convenienza della parallelizzazione dell'esecuzione delle classi tramite *kernel* concorrenti in termini di prestazioni. Il numero di classi latenti è stato fissato a 32, la matrice di occorrenza ha dimensione 1000×5000 e la percentuale di elementi non nulli è stata mantenuta all'1%. La Figura 3.8b mostra il tempo totale di esecuzione, dal quale si evince una diminuzione del tempo all'aumentare del numero di *kernel* concorrenti. Tuttavia, le prestazioni si saturano al raggiungimento di 8 *kernel* concorrenti.



(a) Impatto della *sparseness* sui tempi di esecuzione



(b) Tempo di esecuzione ridotto tramite *current kernel*

Figura 3.8: (a) e (b) mostrano l'impatto della *sparseness* e delle ottimizzazioni tramite *current kernel* sui tempi di esecuzione [22]

Per i dati reali, sono state eseguite entrambe le versioni, CPU e GPU, di PLSI variando il numero di classi latenti da 8 a 50. I tempi di esecuzione, tutti in millisecondi, sono riassunti nelle Tabelle 3.3, 3.4 e 3.5. La Figura 3.9 mostra l'accelerazione della versione GPU rispetto a quella CPU, evidenziando come la versione GPU risulti considerevolmente più veloce. Per quanto riguarda l'esecuzione concorrente dei *kernel*, a causa di un utilizzo inefficiente della memoria globale, non è stata possibile l'esecuzione simultanea di due o più *kernel* per i dataset MED e CISI (Tabelle 3.3 e 3.4). Questo comporta prestazioni leggermente inferiori rispetto

alla versione GPU non concorrente, a causa dell'overhead legato al monitoraggio dello stato dei *kernel* concorrenti. Tuttavia, con il dataset CRAN, è stata possibile l'esecuzione simultanea di 4 *kernel*, ottenendo prestazioni migliori rispetto alla versione GPU non concorrente.

Latent Class	CPU[ms]	GPU [ms]	GPU w/concurrent *kernel* exec.[msl]
4	907.67	30.89	31.31
8	3148.67	59.00	60.30
16	8261.67	116.66	118.34
32	19076.70	223.54	228.51
50	36209.70	349.33	357.80

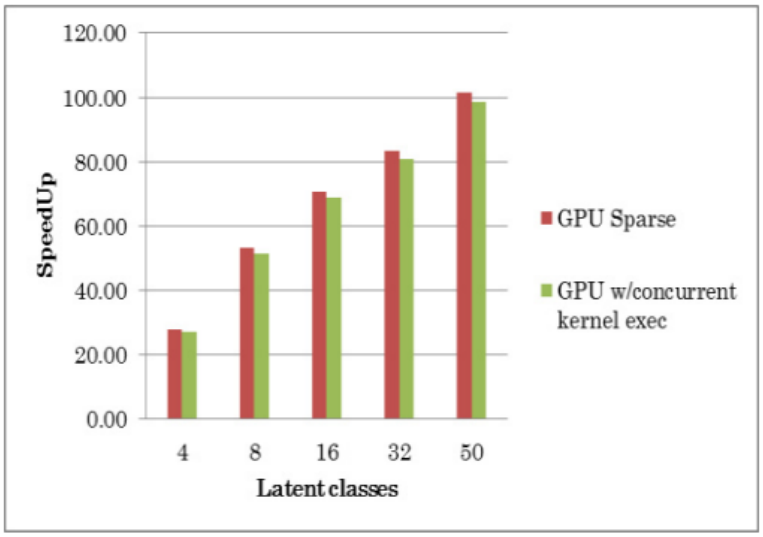
Tabella 3.3: Risultati sperimentali del dataset MED [22]

Latent Class	CPU[ms]	GPU[ms]	GPU w/concurrent *kernel* exec.[ms]
4	646.33	23.09	23.70
8	2357.67	44.21	45.69
16	6155.33	86.95	89.51
32	13967.00	167.73	172.59
50	26575.70	262.04	270.00

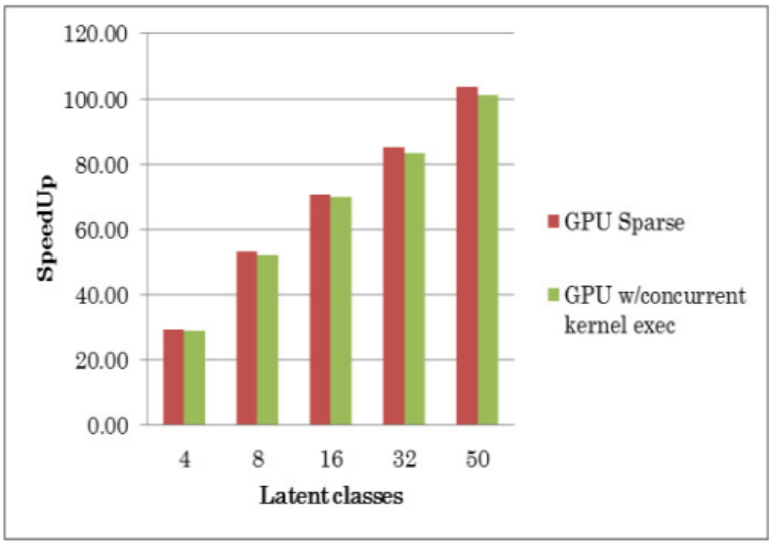
Tabella 3.4: Risultati sperimentali del dataset CISI [22]

Latent Class	CPU[ms]	GPU Sparse[ms]	GPU w/concurrent *kernel* exec.[ms]
4	511.00	31.88	23.77
8	2022.33	60.50	44.83
16	5673.00	119.21	88.26
32	13384.30	232.09	169.91
50	25685.00	362.77	267.21

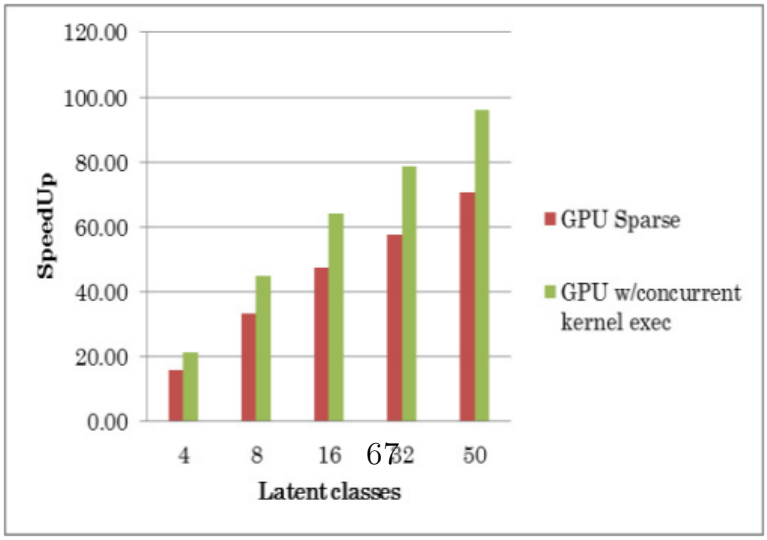
Tabella 3.5: Risultati sperimentali del dataset CRAN [22]



(a) MED dataset



(b) CISI dataset



(c) CRAN dataset

Figura 3.9: [22]

Bibliografia

[1] DeepLearning.AI, Younes Bensouda Mourri, Łukasz Kaiser, and Eddy Shyu. Natural language processing specialization, 2021. Accessed: 2025-09-04.

[2] Wikipedia contributors. Vector space model — Wikipedia, the free encyclopedia, 2025. [Online; accessed 4-September-2025].

[3] Wikipedia contributors. Probability space. `https://en.wikipedia.org/wiki/Probability_space`, 2025. Accessed: 2025-09-02.

[4] Wikipedia contributors. Probability distribution, 2025. Accessed: 2025-09-02.

[5] Wikipedia contributors. Multinomial distribution. `https://en.wikipedia.org/wiki/Multinomial_distribution`, 2025. Accessed: 2025-09-04.

[6] Wikipedia contributors. Joint probability distribution — Wikipedia, the free encyclopedia, 2025. [Online; accessed 4-September-2025].

[7] Wikipedia contributors. Conditional probability — Wikipedia, the free encyclopedia, 2025. [Online; accessed 4-September-2025].

[8] IBM. Che cos'è un modello generativo? `https://www.ibm.com/it-it/think/topics/generative-model`, 2025. Accessed: 2025-09-04.

[9] IBM. Cos'è l'apprendimento non supervisionato? `https://www.ibm.com/it-it/think/topics/unsupervised-learning`, 2021. Accessed: 2025-09-04.

[10] Wikipedia. Teoria dell'apprendimento statistico — wikipedia, l'enciclopedia libera, 2024. [Online; in data 4-settembre-2025].

[11] Carl Eckart and Gale Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.

[12] Thomas Hofmann. Probabilistic latent semantic indexing. In *Proceedings of the 22nd annual international ACM SIGIR conference on Research and development in information retrieval*, pages 50–57, 1999.

[13] Thomas Hofmann. Unsupervised learning by probabilistic latent semantic analysis. *Machine learning*, 42(1):177–196, 2001.

[14] Thomas Hofmann. Probabilistic latent semantic analysis. *arXiv preprint arXiv:1301.6705*, 2013.

[15] Liangjie Hong. A tutorial on probabilistic latent semantic analysis. *arXiv preprint arXiv:1212.3900*, 2012.

[16] Dan Oneata. Probabilistic latent semantic analysis. In *Proceedings of the Fifteenth conference on Uncertainty*, pages 1–7, 1999.

[17] Josef Sivic, Bryan C Russell, Alexei A Efros, Andrew Zisserman, and William T Freeman. Discovering object categories in image collections. 2005.

[18] Florent Monay and Daniel Gatica-Perez. On image auto-annotation with latent space models. In *Proceedings of the eleventh ACM international conference on Multimedia*, pages 275–278, 2003.

[19] Florent Monay and Daniel Gatica-Perez. Plsa-based image auto-annotation: constraining the latent space. In *Proceedings of the 12th annual ACM international conference on Multimedia*, pages 348–351, 2004.

[20] Ren Wu, Bin Zhang, and Meichun Hsu. Clustering billions of data points using gpus. In *Proceedings of the combined workshops on UnConventional high performance computing workshop plus memory access workshop*, pages 1–6, 2009.

[21] John D Owens, David Luebke, Naga Govindaraju, Mark Harris, Jens Krüger, Aaron E Lefohn, and Timothy J Purcell. A survey of general-purpose computation on graphics hardware. In *Computer graphics forum*, volume 26, pages 80–113. Wiley Online Library, 2007.

[22] Eli Koffi Kouassi, Toshiyuki Amagasa, and Hiroyuki Kitagawa. Efficient probabilistic latent semantic indexing using graphics processing unit. *Procedia Computer Science*, 4:382–391, 2011.

Ringraziamenti

Desidero esprimere la mia sincera gratitudine a tutte le persone che hanno contribuito a rendere possibile questo traguardo.

Un grazie speciale va alla mia relatrice, la Prof.ssa Nicoletta Del Buono, per la disponibilità, la pazienza e la costante attenzione con cui mi ha accompagnato lungo il percorso di stesura di questa tesi.

Un pensiero colmo di riconoscenza va alla mia famiglia, che con il proprio sostegno mi ha incoraggiato a credere sempre in me stesso. Grazie per avermi mostrato il vostro orgoglio, per aver riposto in me la vostra fiducia e per avermi concesso la libertà di seguire le mie scelte, rendendo questo traguardo ancora più significativo.

Infine, il ringraziamento più intimo è per Donatella, la mia compagna di vita. Da otto anni è al mio fianco e, soprattutto negli anni universitari, ha reso questo cammino più sereno e leggero. Nei momenti di difficoltà non ha mai fatto mancare un abbraccio, una parola di conforto o un aiuto concreto. A lei va il mio grazie più sentito.

A tutti voi, rivolgo il mio più sentito grazie.